

基于 MSPM0G3507-Eval Board 原神板的 TI MSPM0 芯片开发指南

该文档适用于所有的 TI MSPM0 系列芯片，注意是 MSPM0 才完全适配本文档，如果是传统的 TI 的 MSP430 或者 C2000 之类的芯片是无法按照文档利用 Keil 开发 IDE 进行开发的，请注意甄别！文档着重以自制的 MSPM0G3507 Eval-Board 核心板进行讲解。如果是 TI 的 Launchpad，那么请自行酌情提取关键信息。

一、核心板介绍

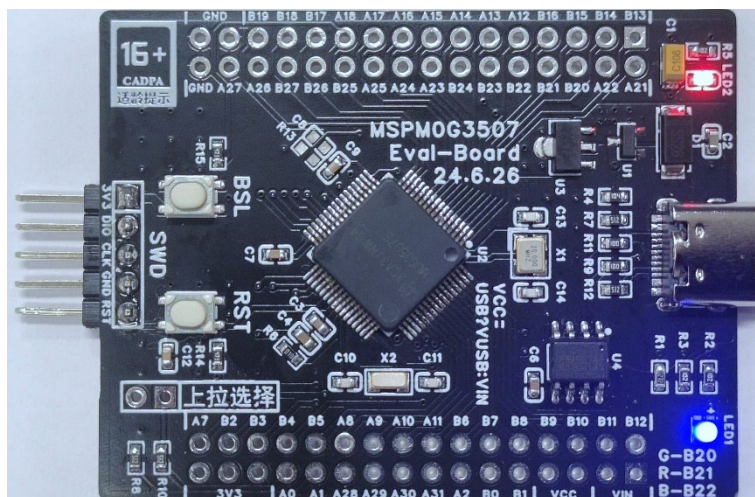
核心板以 TI MSPM0G3507 SPMR 为核心芯片，MSPM0G3507 芯片所具备的功能和特性如下：

- Arm® 32 位 Cortex® M0+ CPU，主频最高 80MHz，128KB Flash，32KB SRAM
- 1.62V 至 3.6V 工作电源电压
- 多达 60 个 GPIO
- 7 个定时器支持多达 22 个 PWM 通道
- 具有报警和日历模式的 RTC
- 4 个硬件 UART 接口、2 个硬件 I2C 接口、2 个硬件 SPI 接口、1 个硬件 CAN 接口
- 2 个 12 位 4Msps 同步采样 ADC，支持 17 个外部通道
- 1 个 12 位 1Msps 的 DAC，集成输出缓冲器
- 2 个零漂移、零交叉斩波运算放大器
- 1 个通用放大器
- 3 个具有 8 位基准 DAC 的高速比较器
- ADC、DAC、运放和比较器可以编程配置内部相连

由于该 MCU 芯片接口丰富的同时直接集成了 DAC、运放、比较器，这是相较于其他常用的 MCU 例如 STM32F103 比较突出特有的，并且同样可以跟 STM32 一样用 HAL 硬件抽象的方式编程开发，此外还是基于 Arm 的内核设计的（这意味着能够和 STM32、GD32 之类以 Arm 为内核的 MCU 一样使用 Keil 以及 DAP-

CMSIS 调试器开发),更有在电赛中用 TI 为核心芯片会是某些题目的硬性要求,没有硬性要求的题目使用 TI 的 MCU 会有一定的分数加成的规矩,因此 MSPM0G3507 纳入为我们电赛时使用的单片机平台。

核心板集成了 USB Type-C 串口、SWD 调试口、外接晶振、RTC 晶振、3.3V 稳压供电、串口下载按键 (BSL)、重置按键 (RST)、RGB 灯、电源灯及全引出 GPIO 等部件,可完整地使用 MSPM0G3507 的所有功能。



1.1 供电

该开发板的供电方式有 3 种, USB 供电、VIN 供电 4.7V~10V (一般建议 5V)、直接在 3V3 管脚供电 3.3V。**一般建议用 5V 电压通过 VIN 系统供电**,此时系统会自动根据是否插入 USB 线来切换系统的供电来源,如果有插入 USB 线,则通过 USB 线供电,否则通过 VIN 供电,此举在通过电池提供 5V 给板子供电时有节能作用。正常供电的情况下,红灯 LED2 会点亮。

1.1.1 通过 USB Type-C 供电

通过常见的例如电脑或者手机充电器进行 USB 供电,此时相当于通过 USB 口用 5V 给系统供电。假如此时系统还通过 VIN 供电,并且 $VIN \leq 6V$,那么此时输出 VCC 的来源是 USB 的 5V;如果 $VIN \geq 6V$,那么 VCC 的来源依旧是 VIN。

1.1.2 通过 VIN 供电

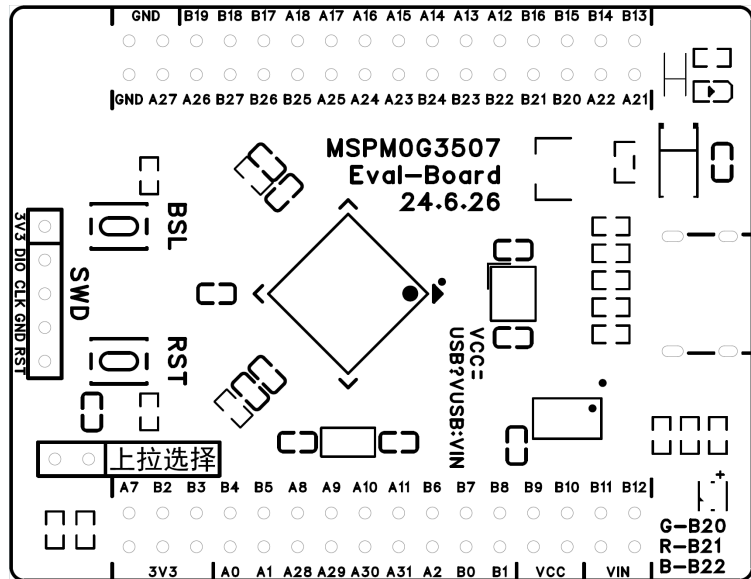
VIN 管脚供以 4.7V~10V 的电压即可,但是建议是 5V。VIN 比 5V 高的越多,无用的发热越严重。

1.1.3 通过 3V3 供电

在 3V3 管脚上施加 3.3V 的驱动电压,同样可以让 MCU 工作起来。

1.2 全管脚 GPIO

核心板把 MSPM0G3507 芯片的 LQFP-64 封装的所有引出的可用管脚都引出到了管脚上，管脚定义直接标在了核心板的管脚边上，如果模糊看不清，可以参考下图



关于 GPIO 的注意事项如下：

- BSL 按键所对应的管脚是 PA18
- PA0、PA1 为 OD 开漏管脚，如果需要按照 3.3V 的逻辑电平进行使用，需要如图红框将“上拉选择”排针用跳线帽连接起来，否则这两个管脚无法输出高电平。如果需要用别的电平进行上拉，以匹配其他 MCU 的逻辑电平，则将该跳线帽取下，在如图的蓝圈的管脚上加上相应的电压，其可接受的上拉电压范围为 1.62V~5V



- RGB 灯内部直接与 PB20、PB21、PB22 连接了，所以可以直接通过相应的排针管脚与 RGB 灯交互

1.3 板载 RGB 灯

核心板上自带一个 RGB 灯，红灯驱动管脚内部连接到 PB21，绿灯驱动管脚内部连接到 PB20，蓝灯管脚内部连接到 PB22，三者皆为低电平有效。同时排针

管脚上又有 PB20、PB21 和 PB22，因此其实也可以通过将 GND 连接到该三管脚确认 RGB 灯是否能够正常工作。



1.4 USB 串口

核心板板载了 USB 转串口芯片，其 USB Type-C 口通过该芯片与 G3507 的 UART0 连接。同时由于该芯片支持串口烧录固件程序，类似 STM32 的 Hex 固件烧录，可以快速地下载打包好的固件程序，默认烧录程序的串口为 UART0，因此可以仅仅通过 USB Type-C 线以及烧录工具，即可完成芯片固件的烧写。

二、开发工具链方案

2.1 相关软件简介

我们下发的 TI MSPM0G3507 核心板，开发相关的软件有：TI 的 Code Composer Studio（简称 CCS）、TI 的 System Configuration Tool（简称 SysConfig）、TI 的 MSPM0 SDK（简称 M0SDK）以及 Keil uvision5（简称 Keil）。

CCS 是 TI 官方针对自己的芯片开发的 IDE 软件，内部包含 TI 的芯片的开发工具库，其功能类似于 Keil，但是是 TI 芯片的特供。由于 CCS 只支持 TI 自家的昂贵的 XDS 调试器以及同样价格不菲的 J-Link 调试器，因此如果没有 XDS 或者 J-Link 调试器，或者没有自带 XDS 调试器的开发板例如从 TI 申请的 Launchpad，那么是无法利用 CCS 软件进行代码的硬件 Debug 调试的。

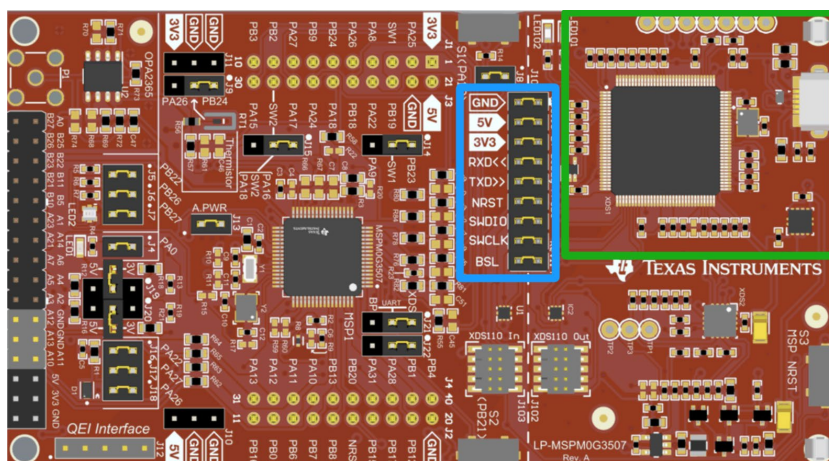
SysConfig 是 TI 配套的配置工具，正如 STM32 的 CubeMX 一样，旨在简化芯片开发中对硬件和软件功能的配置，用于配置引脚、外设、无线电、软件栈、RTOS、时钟树和其他元件等功能，自动化生成芯片相关功能的定义和初始化函数，生成硬件抽象层 HAL 库，简化开发工作。该软件可以单独安装，同时也集成在了 CCS 中。

M0SDK 包提供了 TI MSPM0 所有芯片的库、配置工具脚本、参考文档以及丰富的例程，例程提供了基于 CCS、Keil、IAR 等开发 IDE 的项目工程，例程涵盖了芯片绝大部分功能的基本使用方法。

Keil 是很常用的单片机开发 IDE，可以开发 51 单片机、STM32 等各种各样的单片机。由于 TI MSPM0 系列的 MCU 是基于 ARM Cortex M0+内核的单片机，因此可以与 STM32 之类的基于 ARM 内核的 MCU 一样，借助 Keil、DAP CMSIS 调试器进行硬件调试开发。正因如此，在合适的配置下，同样可以在没有 TI 的昂贵的 XDS 调试器无法利用 CCS 调试开发的情况下，同样进行硬件调试开发。

此外，还有一些额外的小工具和脚本，可以帮助我们更加自由地选择如何开发 TI MSPM0 芯片。此文涉及到的工具和脚本有：BSL 固件下载工具（利用串口直接向 MCU 烧录 BSL 固件镜像文件），SysConfig 命令行执行脚本（以命令行方式执行 SysConfig 从而自动生成库文件和初始化代码）以及 BSL 固件转换脚本 gentxt 脚本（将 Keil 生成的 Hex 镜像文件转换成 TI 的 txt 格式 BSL 镜像文件）。

再次提醒，如果是 TI 的相关芯片的 LaunchPad，例如下图所示的 MSPM0G3507 LaunchPad，由于所有的 TI 的 Launchpad 都有**绿框部分**的板载 XDS 调试器，因此建议直接 CCS 软件去进行调试和开发就够了——代码层面一模一样，撑死就是 IDE 界面和使用步骤有一点点小差别，包稳的。对 LaunchPad 的使用不在本文档的介绍范围内，但是还是要提醒一下，以 G3507 LaunchPad 为例，其上的 microUSB 口同时具备 XDS 调试器功能以及 G3507 的 UART0 的串口 USB 功能，因此插上板子后会在系统里识别出两个端口号，此点从 LaunchPad 右侧和左侧 G3507 开发板部分通过**蓝框**里的跳线帽连接的有 TXD、RXD 以及 SWD 接口即可得知。这也意味着如果把跳线帽全部摘掉，那么最半边的板子就可以跟我们的核心板一样去看待和使用。**另外 LaunchPad 也可以用 Keil 软件开发，调试器选择成 CMSIS DAP 即可，但是非常不建议！将 LaunchPad 用 Keil 以 CMSIS DAP 进行调试无法正确启用断点！！**



以下列举针对我们的 MSPM0G3507 核心板的三种开发环境的配置方案，可以根据实际情况自行选择。

2.2 方案一：CCS+BSL 固件下载工具

CSS 软件中就内置了 SysConfig，因此仅仅安装 CCS 以及芯片的软件开发套件 M0SDK 就可以进行开发。但是正如前文所述，CCS 只支持 TI 自己的以及 J-Link 调试器，我们并没有配备 TI 或者 J-Link 调试器，因此在不额外购买的情况下，只能利用 CCS 写代码，设置代码项目生成 txt 后缀格式的固件文件，再配合 BSL 固件下载工具，将程序下载到芯片中，无法进行硬件在线调试。这种方式适合不怎么需要硬件在线调试的开发需求。

优势：环境配置简单

劣势：无法硬件断点调试

所需下载安装软件：CCS、M0SDK、BSL 固件下载工具。

所需硬件：USB Type-C 数据线

2.2 方案二：SysConfig+Keil

单独安装 SysConfig，配合 M0SDK 中自带的相关脚本和工具，对 Keil 进行一些配置，从而可以通过 Keil 软件直接调用 SysConfig 配置工具，自动生成库文件及初始化函数代码。后续的代码撰写和硬件调试操作步骤和基于 CMSIS DAP 调试器调试 STM32 一致，并且同样可以通过 Keil 里的“load”功能将程序烧写到芯片中。

优势：可以与 STM32 等其他利用 Keil 开发的场景复用，不需要额外下载 CCS 这种大软件

劣势：需要对 Keil 软件进行一系列配置，少许有点麻烦，且在 Keil 的项目同样需要进行一系列的配置（Keil 软件配置一次一劳永逸，项目配置每次都要，但是可以通过利用模板项目进行修改以省去每次的配置）

所需下载安装的软件：SysConfig、Keil、M0SDK

所需硬件：Pwlink2 lite 调试器

2.3 方案三：SysConfig+Keil+BSL 固件下载工具+gentxt 脚本（本文主推）

本方案其实就是在方案二的基础上，再额外添加一个脚本工具，在项目多配置一项处理参数，使得保有方案二的功能之外，同时还能生成可快速烧录下载

的 BSL 固件，就类似开发 STM32 时勾选上生成 Hex 镜像文件一样，多加了一个将程序打包生成 TI 的 MCU 所支持的以 txt 为文件格式的 BSL 程序固件镜像。

优势：形成了基于 Keil 开发 TI 的 MSPM0 芯片的完整工具链

劣势：多加一步配置，但是和方案二一样，可以重复利用一个模板以解决

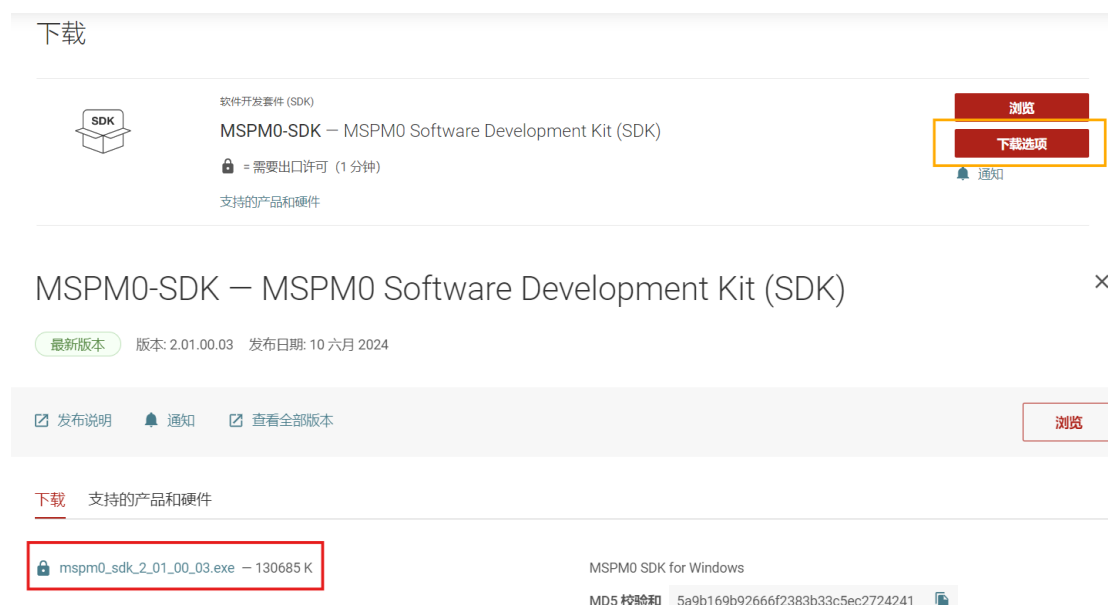
所需下载安装的软件：SysConfig、Keil、M0SDK、BSL 固件下载工具、gentxt 程序及调用脚本

三、软件安装配置

这里把所有的方案设计的软件安装和配置进行讲解，注意安装顺序最好按照本章节进行，特别是对于方案二和三，否则会导致某些步骤无法正常进行！

3.1 MSPM0 SDK 安装

前往 <https://www.ti.com.cn/tool/cn/MSPM0-SDK#downloads>，点击“下载选项”，



在“下载选项”中点击“mspm0_sdk_x_xx_xx_xx.exe”，但是需要注册账号，注册账号后登录再点击，会进入一个提交申请表格的页面，前边随便填，email 填自己的邮箱，以及一定要选“civil”不能选“military”，然后再往下拉即可下载。

To download or access:

- Certain Software/Tools/Documents require export approval before download or access.
- If you are approved, a DOWNLOAD BUTTON will appear.
- If you are not immediately approved, a message will appear after this form; Software may be delayed 1-2+ business days.
- To AVOID delays, please provide complete information.

U.S. Government export approval:

All fields are Required. Incomplete information will be DENIED.

First name:

Last name:

Your email address:

Your full company/university name:

Country this file will be used in:

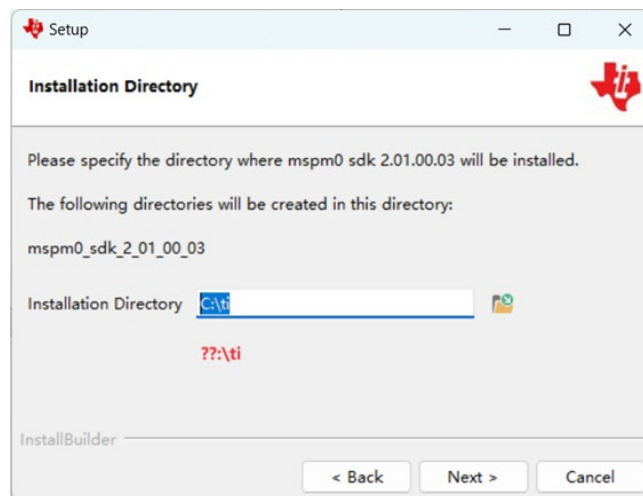
What end-equipment/application will you use this file for:

☐ Military

☐ Civil

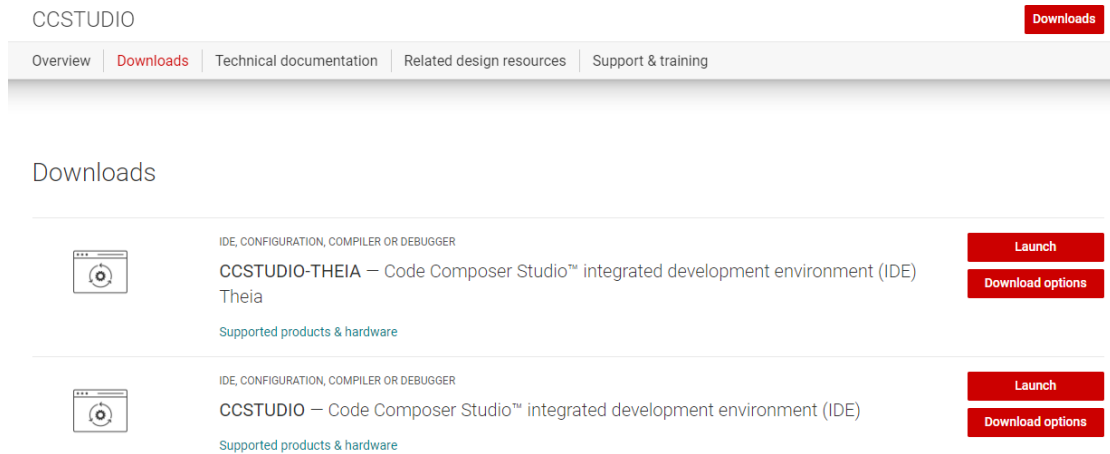
如果不想注册，可以到 <ftp://10.92.4.88/电赛培训/软件包/TI 相关/MSPM0> 中下载，文档成文时是 2.01.00.03 版本。

安装过程中路径可以改，但是建议保证是“?:\ti”，当然也可以自己找个地方安装，后续对照着自行变更即可。本文所有截图以默认安装的位置 C:\ti 为例进行讲解。接下来会进入很久的解包 Unpack 过程，耐心等待，文件很多会挺久的。等待完成即可。

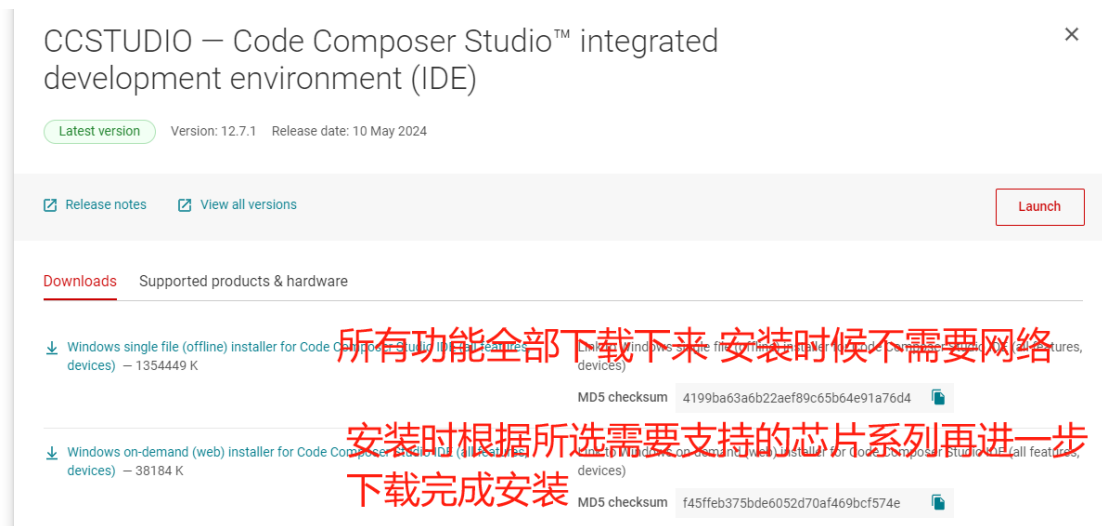


3.2 Code Composer Studio 安装和配置

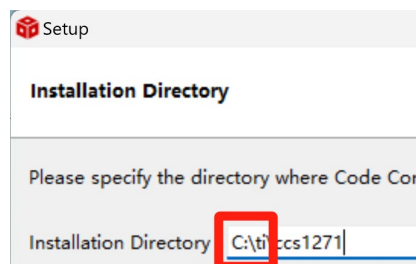
前往 <https://www.ti.com/tool/CCSTUDIO> 页面，会发现有两个 CCS 可供下载。其中带 THEIA 的是基于 Eclipse Theia 框架平台开发的，很多开发 IDE 如 Arduino IDE 等正在逐渐往这个平台上迁移，是今年刚推出的比较新，可能存在未知的 bug，但是未来的 CCS 软件也都将是这个框架，喜欢尝鲜的可以下载这个版本，风格有点类似 VSCode；没带的是传统的原生基于 Eclipse 开发的 CCS。本文以截止发文时最新的明确可靠稳定的 CCS12.7.1 为例进行讲解。



这里下载了 **offline** 离线版本，当然也可以直接从 <ftp://10.92.4.88/电赛培训/软件包/TI 相关> 中下载离线版本。

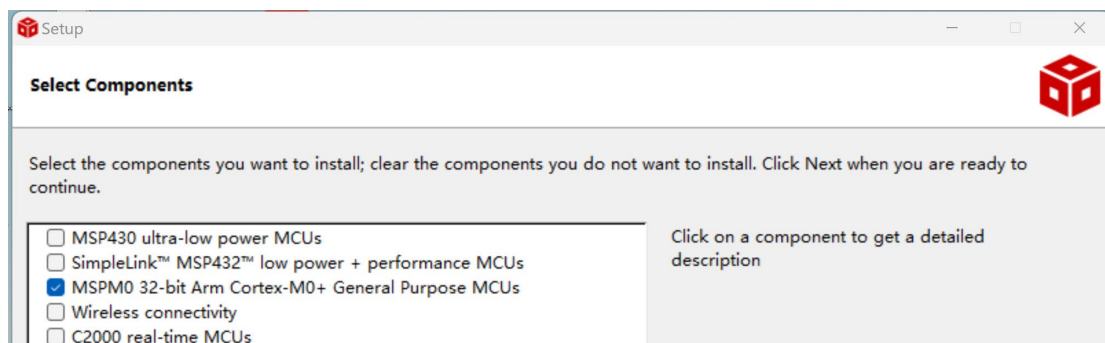


下载解压缩后直接双击“**ccs_setup_12.7.1.00001.exe**”开始安装，在选择安装路径位置这一步，确保安装的位置前缀与上一步安装 **M0SDK** 相同，如下图所示，我的安装目录除去最后的 **ccs1271**，前边保证了和之前 **M0SDK** 的安装位置一致，这非常重要，可以省去很多不必要的麻烦。

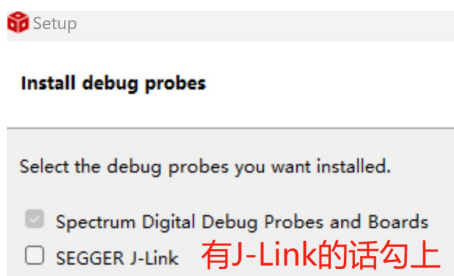


接下来按照他的推荐选择“**Custom Installation**”，选择将要支持的芯片类型，这里如果手上有其他的系列的 **TI** 的 **MCU** 芯片，可以勾选上合适的产品选项，

例如你还有 TI 的 MSP430 芯片开发，那么就勾选上“MSP430.....”这一项。
此处我们仅考虑支持 MSPM0 系列芯片（因为我们是 MSPM0G3507）。



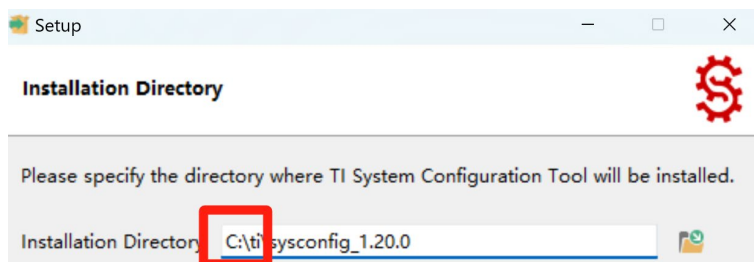
调试器支持选择界面，如果你有 J-Link，那么记得勾选上“SEGGER J-Link”。



后续一路 Next 下一步然后等待安装完成即可。安装结束需要重启一次。

3.3 SysConfig 安装

前往 <https://www.ti.com/tool/SYSCONFIG> 进行下载。同样可以到也可以直接从 <ftp://10.92.4.88/电赛培训/软件包/TI> 相关 中下载。下载完成之后直接双击打开开始安装，同样，需要保证期安装目录除开最后的 sysconfig_1.20.0 前边与之前安装 M0SDK 的目录一致！



接下来一路 Next 让他安装完成即可。

3.4 Keil uVision5 安装和配置

这个没啥好说，应该到处都有参考。此处着重讲安装之后的配置工作。

3.4.1 开发固件包下载安装

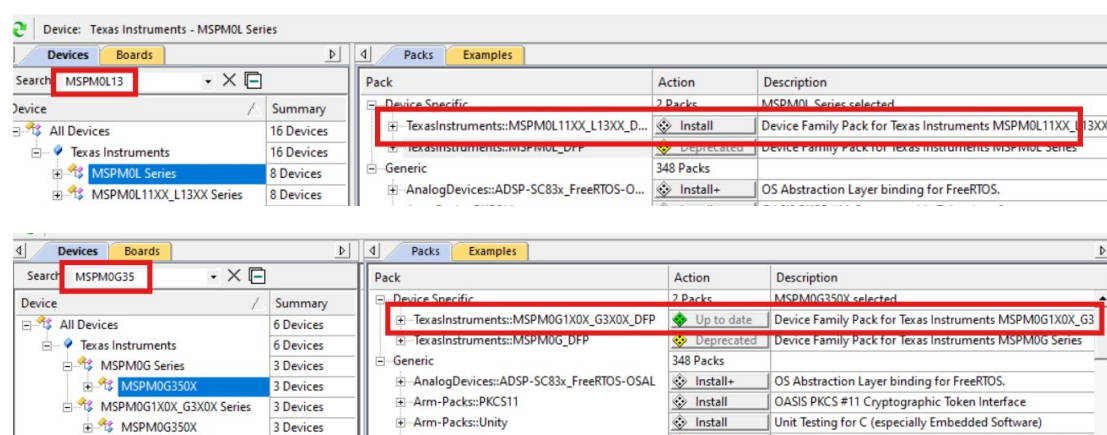
MSPM0 的 Keil 固件包安装但是安装完之后，需要下载 TIMSPM0 的开发固

件包 DFP。

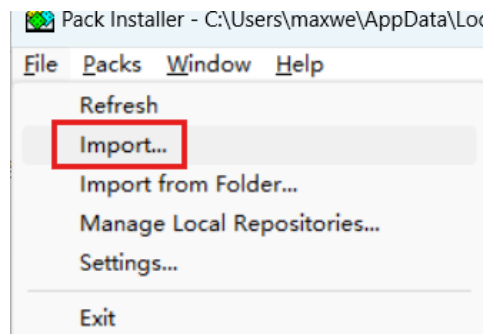
(i)如果有全局翻墙的手段,可以直接借助 Keil 的 Pack Installer 进行安装。
步骤如下:先启用全局翻墙,然后打开 Keil 时 Pack Installer 会自动弹出,或者手动点击调取



在 Pack Installer 等待其完成自动更新等操作完成后,在列表中搜索相关的芯片型号,例如最常用的 MSPM0L1306 和 MSPM0G3507,点选到相应的细分项,一般最顶上的能够有“Install”按键的,就是需要安装的固件包。自己也注意看一下相关的固件包后的 Description 描述,截止撰写日 2024 年 6 月 19 日,L01306 需安装 MSPM0L11XX_L13XX_DFP,G3507 需安装 MSPM0G1X0X_G3X0X_DFP 固件包。

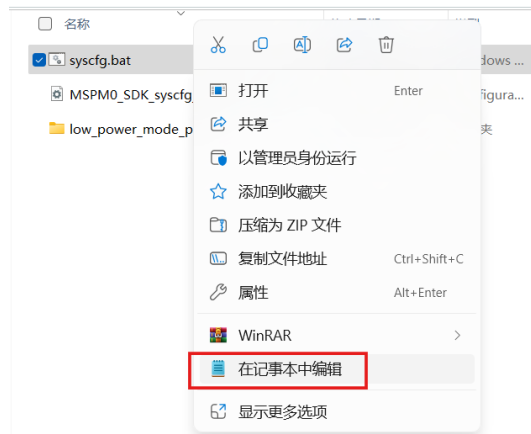


(ii) 为了避免下载速度太慢,可以直接就从 <ftp://10.92.4.88/电赛培训/软件包/TI 相关/Keil 中固件包>或者自行到 <https://www.keil.arm.com/packs/>进行相应的固件包下载,然后在 Keil 的 Pack Installer 中,点击“File->Import”,选择下载好的固件包,即可进行安装。

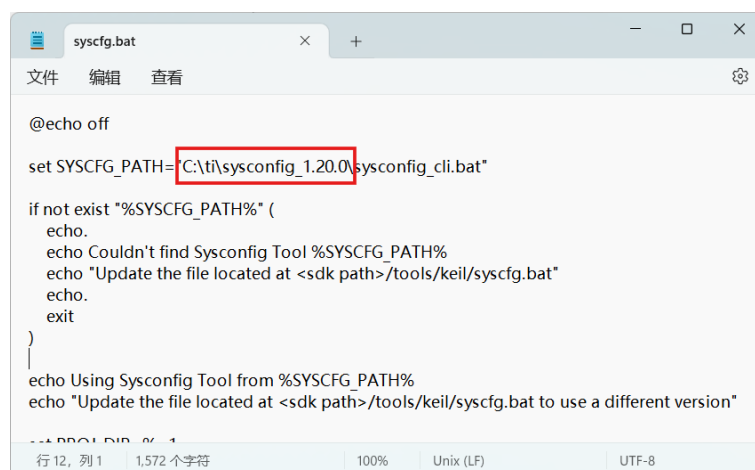


3.4.2 用于 Keil 的 SysConfig 插件添加

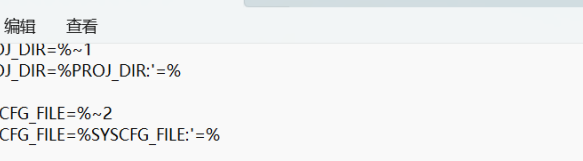
根据自己之前安装的 M0SDK 的位置，进入“M0SDK 安装目录”\tools\keil 文件夹，例如我的是在 C:\ti\mspm0_sdk_2_01_00_03\tools\keil，然后右键“在记事本中编辑”打开 syscfg.bat 进行编辑（不是双击！双击就是执行了）。



修改 SYSCFG_PATH 前半部分路径为你自己安装的 SysConfig 软件的路径，比如我的是 C:\ti\sysconfig_1.20.0，最终是 C:\ti\sysconfig_1.20.0\sysconfig_cli.bat



修改 SDK_ROOT 为你自己安装 M0SDK 的路径，跟前边找这个 syscfg.bat 的路径是一样的，比如我的是 C:\ti\mspm0_sdk_2_01_00_03。注意要用英文的双引号括起来！



```
set PROJ_DIR=%~1
set PROJ_DIR=%PROJ_DIR:='=%

set SYS_CFG_FILE=%~2
set SYS_CFG_FILE=%SYS_CFG_FILE:='=%

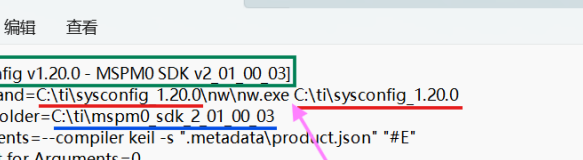
:: Search for the root of the SDK by going up one directory
:: However, if we don't find it after 20 times then give up
set SDK_ROOT="C:\t\mspm0_sdk_2_01_00_03"
set iter=0

:sdk_search_loop
if exist "%SDK_ROOT%\metadata\product.json" (
    goto sdk_search_exit
) else if %iter% geq 20 (
    @echo "Couldn't find .metadata\product.json"
) else (
    set /a iter=%iter%+1
    cd /d %~dp0%..
    goto sdk_search_loop
)

:sdk_search_exit
echo %SDK_ROOT%

exit /b
```

在同一文件夹中，打开 `MSPM0_SDK_syscfg_menu_import.cfg` 进行编辑，如果无法打开，手动指定记事本之类的文本编辑器打开，修改确认修改 `SysConfig` 和 `SDK` 版本与路径。注意红色划线部分为你自己安装 `SysConfig` 软件的路径地址；注意修改 `SysConfig` 路径时，确保紫色箭头所指的位置是有个空格的！蓝色划线部分为你自己安装 `M0SDK` 的路径地址；绿色框内的文本是插件在 `Keil` 软件里显示的菜单名称，虽然无伤大雅，但是也尽量修改成与你实际安装的 `SysConfig` 和 `M0SDK` 版本一致的信息吧，说不定以后会有其他插件，到时候才不会乱套。



文件 编辑 查看

[Sysconfig v1.20.0 - MSPM0 SDK v2_01_00_03]

Command=C:\ti\sysconfig_1.20.0\nw\nw.exe C:\ti\sysconfig_1.20.0

Initial Folder=C:\ti\mspm0_sdk_2_01_00_03

Arguments=--compiler keil -s *.metadata\product.json" "#E"

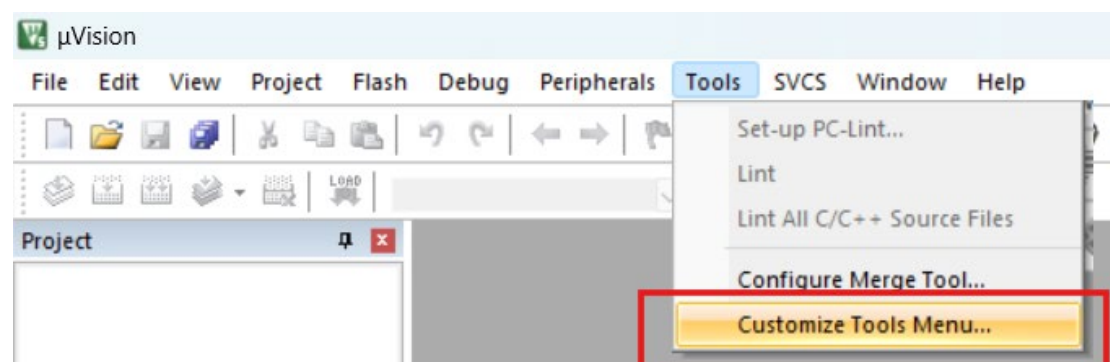
Prompt for Arguments=0

Run Minimized=0

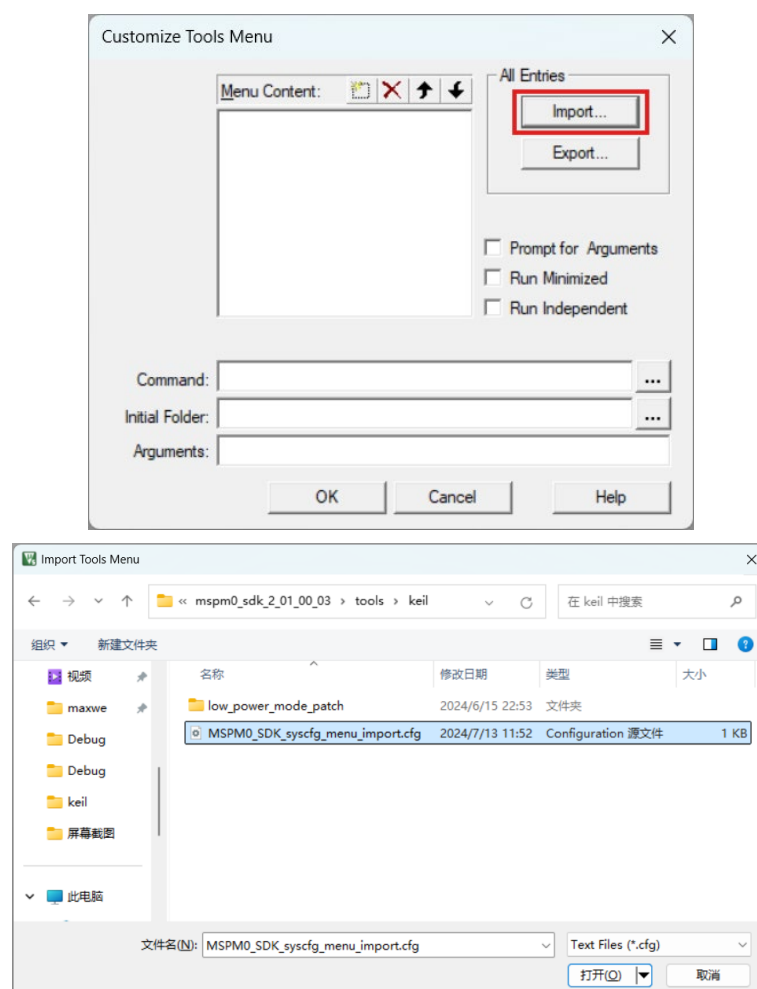
Run Independent=-1

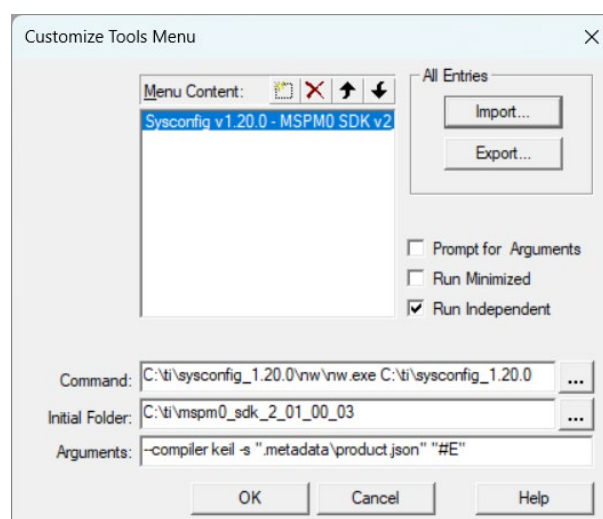
行 8, 列 1 267 个字符 100% Unix (LF) UTF-8

打开 Keil uVision5，选择菜单栏中 Tools->Customize Tools Menu，进入自定义菜单插件设置功能。

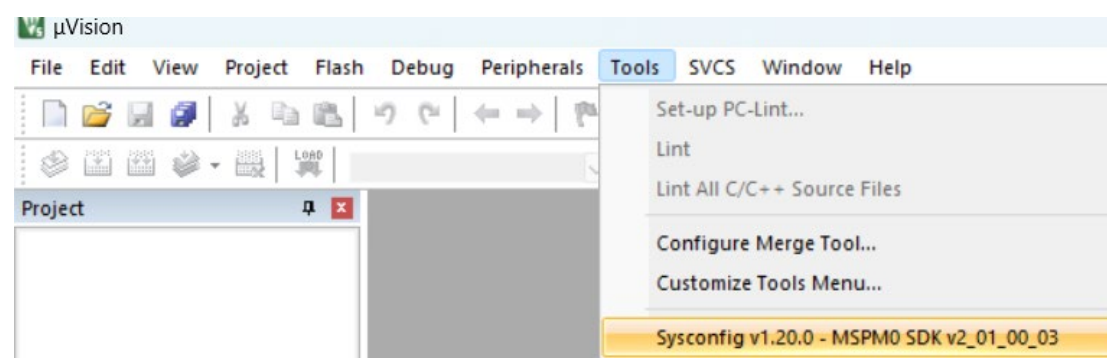


点击 Import，在弹出的文件选择窗口中，进入到刚刚修改 config 文件的目录下，也就是<你安装 M0SDK 的路径>\tools\keil，选择我们刚刚修改完的配置文件 MSPM0_SDK_syscfg_menu_import.cfg，点击“打开”，即可将配置文件中的信息自动导入，可以看到配置文件中的数据自动填入，最后点个“OK”，即可完成 Keil 的 SysConfig 插件菜单的添加。





如果添加成功，那么现在点击菜单栏中的 Tools，就可以看到最后一项就是我们刚刚新添加的 SysConfig 菜单工具了。



3.5 工具脚本下载及配置

这部分只能从 ftp 上下载，从//10.92.4.88/电赛培训/软件包/TI 相关/MSPM0 中下载“BSL_download_tool2”，此乃用于将打包成 TI 芯片可用的 txt 格式的 BSL 镜像文件（也就是 STM32 之类的芯片的 Hex 或者 Bin 固件镜像文件）通过串口烧录下载到 MSPM0 芯片的小工具。

同个目录下，下载“gentxt”文件夹，将整个文件夹同样复制粘贴到 M0SDK 安装的目录下。如果 SysConfig、CCS、M0SDK 都下载安装了的话，此项操作后盖安装目录将如下图，请再次确保所有与 TI 相关的软件安装在同个目录下！！

名称	修改日期	类型
.tirex-temp	2024/6/16 11:46	文件夹
ccs1271	2024/7/26 1:54	文件夹
gentxt	2024/7/14 20:46	文件夹
mspm0_sdk_2_01_00_03	2024/6/15 22:54	文件夹
sysconfig_1.20.0	2024/7/26 2:13	文件夹

同样，编辑 gentxt 中的 gentxt.bat 脚本文件，修改 GENTXT_PATH 中 gentxt 文件夹的根目录地址。

```
@echo off
set GENTXT_PATH="C:\ti\gentxt\tiarmhex.exe"
if not exist "%GENTXT_PATH%" (
    echo.
```

你自己放置gentxt文件夹的目录地址

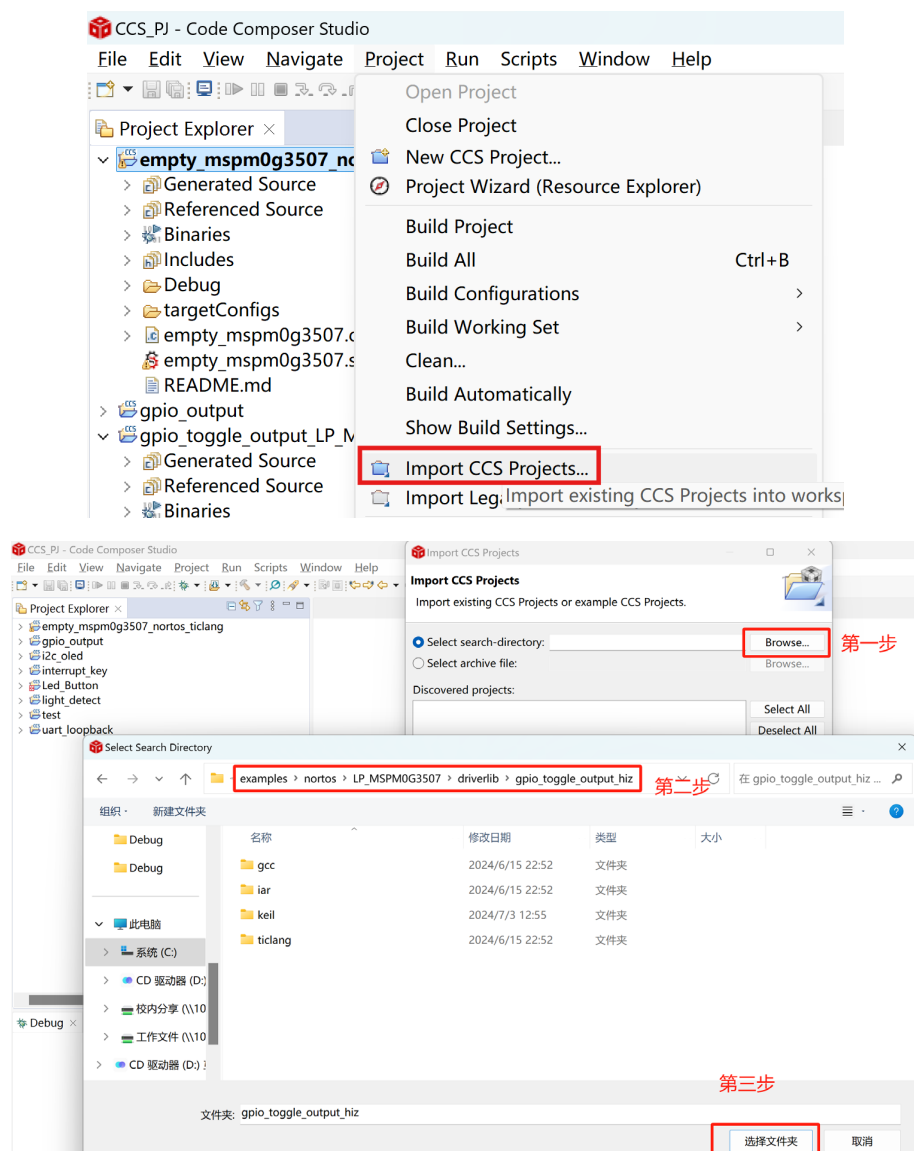
四、基于 CCS 使用 MSPM0G3507 官方例程

从 M0SDK 安装的目录底下，有很多 M0 系列各个芯片的例程，但是都是给 LaunchPad 的，不过除却硬件上可能跟我们的核心板有些差别，例如板载 LED 灯之类的资源管脚不一致之外，其实大部分是可以借鉴的，具体目录为“M0SDK 安装目录”\examples\nortos\LP_MSPM0G3507，基础部分功能可以进一步进入 driverlib 文件夹底下查看。

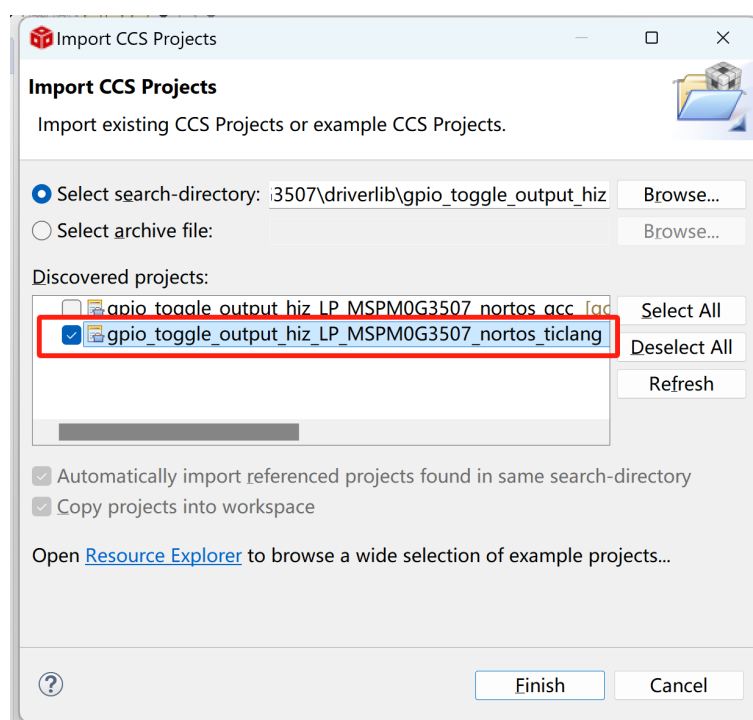
现在简述从 CCS 如何加载并烧录某个例程。

4.1 例程加载及 CCS 配置

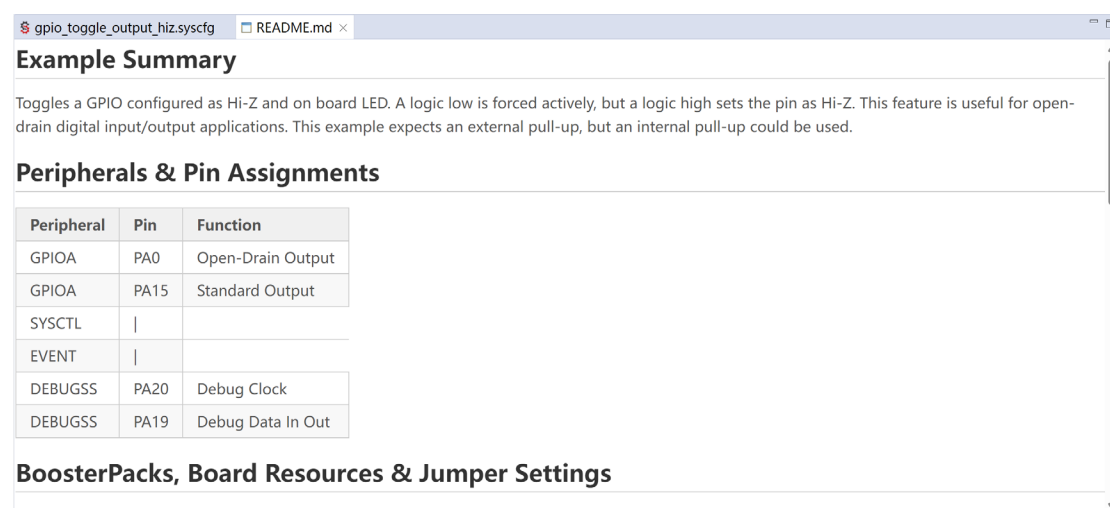
打开 CCS, Project->Import CCS Projects..., 然后选择到某个 M0SDK 的 nortos 的例程的文件夹，假设现在我们要打开 GPIO 控制的例程，



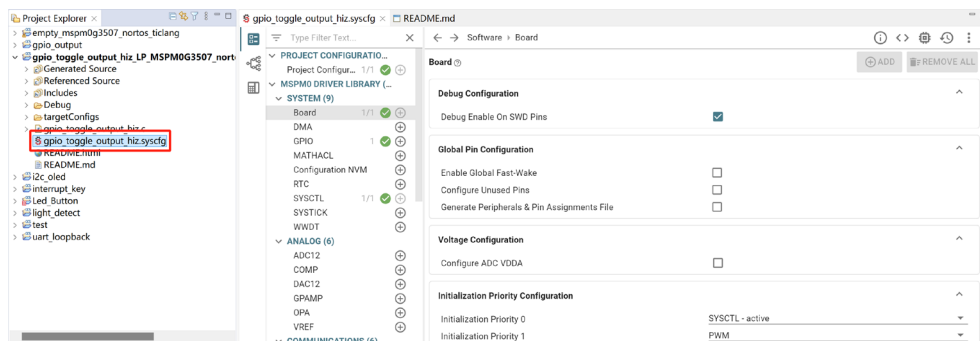
进入 `gpio_toggle_output_hiz` 中，直接点击选择文件夹，那么 CCS 会自动扫描出能够用 CSS 打开的所有项目工程，结果如图所示扫描出来了两个项目工程



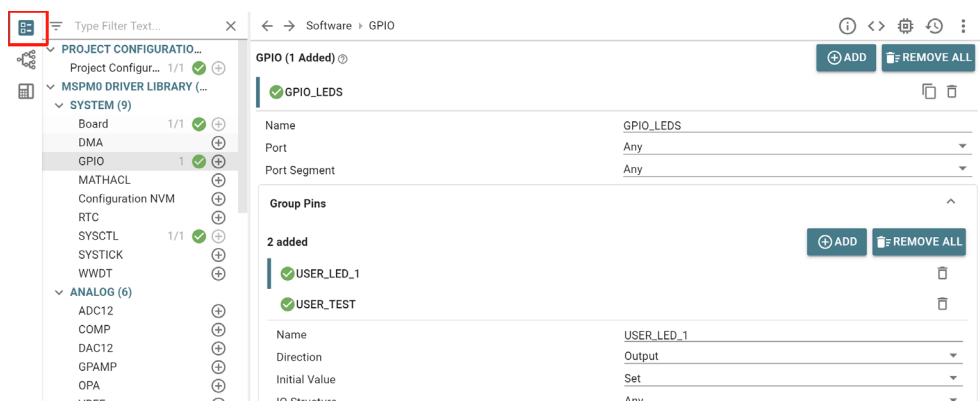
选择最后为 ticlang 的项目，勾选并点击 Finish，那么 CCS 会将该例程导入并重新在你的工作区文件夹里头生成一个新的该例程的项目文件夹副本（也就是原本例程里的文件是不动的）。接下来 CCS 自动进入该项目，并自动把项目中的 README 文档打开，官方的所有例程的都有 README，会告诉你这里头用到了哪些管脚，分别是什么功能。



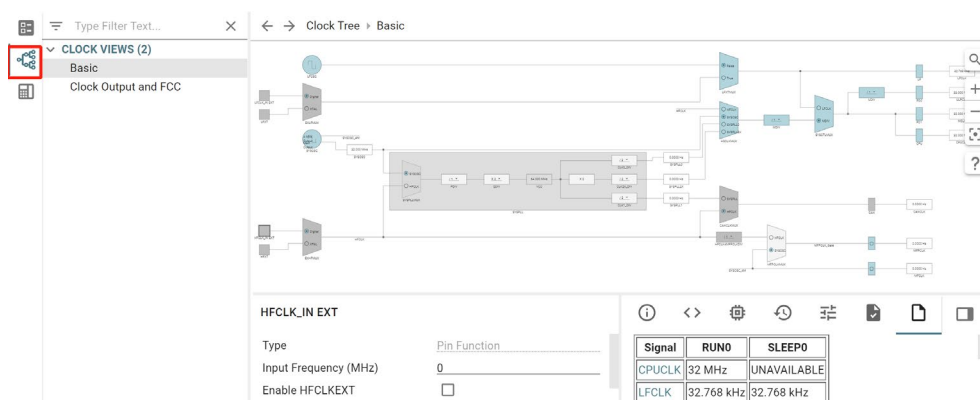
同时打开项目中的 .syscfg 后缀的文件，CCS 会利用内置的 SysConfig 功能直接以可视化配置界面打开该工程配置文件，界面与单独安装的 SysConfig 是一模一样的，功能也是一样的。



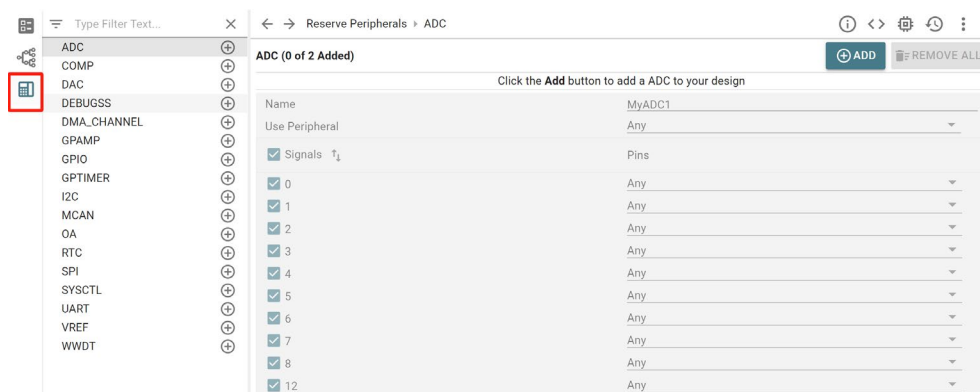
SysConfig 中有三个功能分区，第一个是所有的功能及中断等



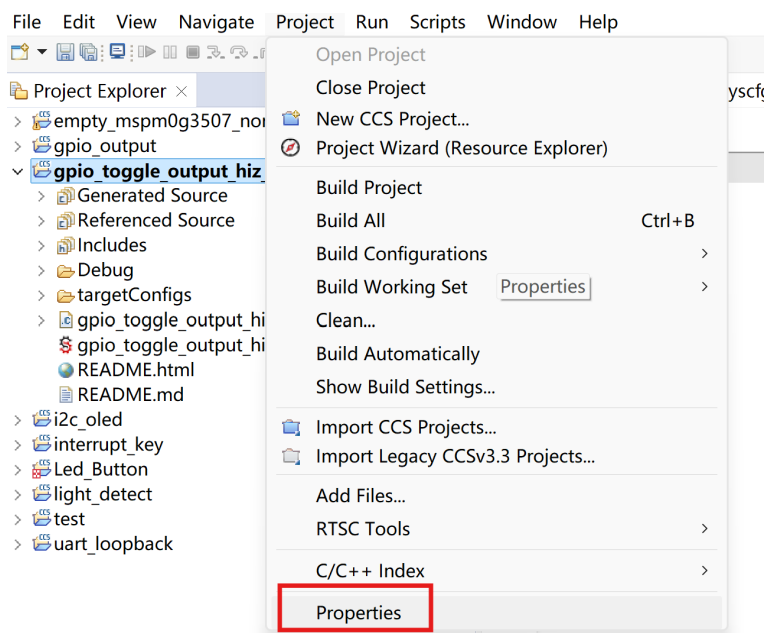
第二个是时钟树



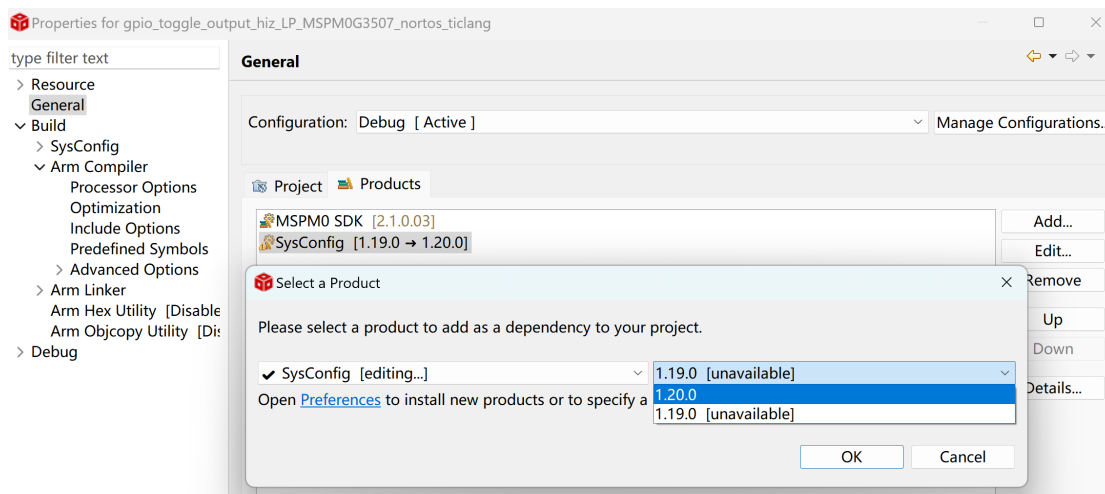
第三个是保留的保留外设的功能设置



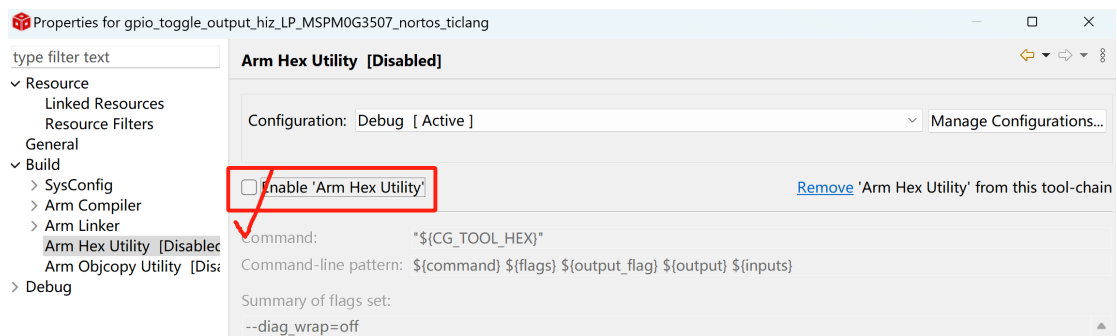
关于 SysConfig 的使用可以简单摸索即可了解，有修改就记得保存即可。接下来需要设置 CCS 使其在成功编译代码之后将程序打包成 TI 芯片所能烧录的文件格式为 txt 的 BSL 固件文件，使得能够后续让程序在我们的核心板上运行，在确保当前项目为 Active，即在左侧多个项目的窗口里（假如你导入过多个项目工程）当前项目加粗，点击 **Project->Property**，进入项目的配置选项界面。



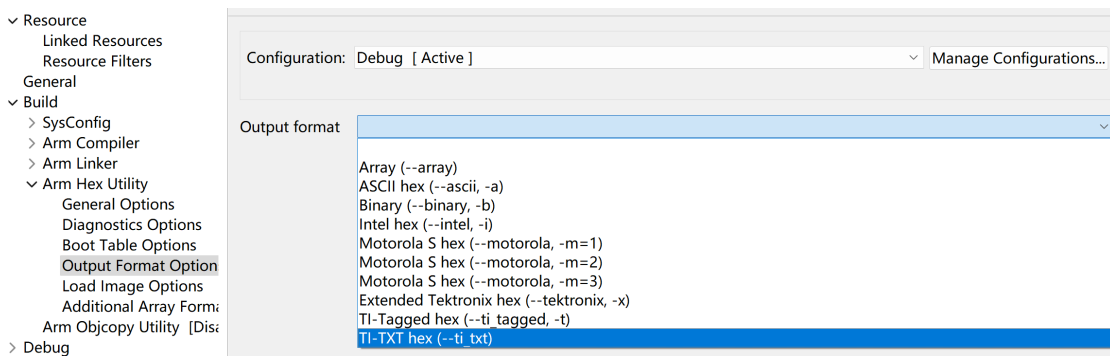
在 **General->Products** 中可以看到，例程项目是基于 MSPM0 SDK 2.1.03 以及 SysConfig1.19.0 开发的，但是我们 SysConfig 安装的版本并不是 1.19.0，因此需要修改选择，双击 SysConfig 条目，在弹出的对话框中，点开右侧下拉框将 SysConfig 版本切换到我们安装的 1.20.0



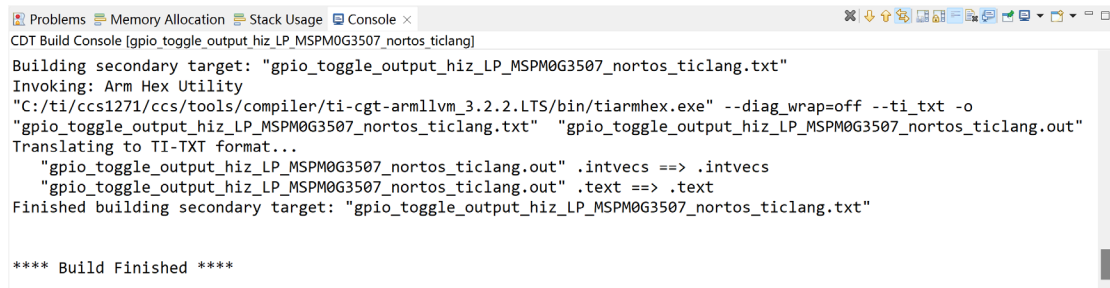
然后进入 Arm Hex Utility 页面，可以看到例程是将此功能关闭了的，此处我们要将其 Enable 开启



接着该项目才会展开子项目，在 Output Format Option 子页面中，打开 Output format 下拉框，选择到“TI-TXT hex”项目，这项设置使得代码成功编译之后生成能够利用 BSL 下载工具下载到芯片中的程序镜像文件，文件格式为 txt。



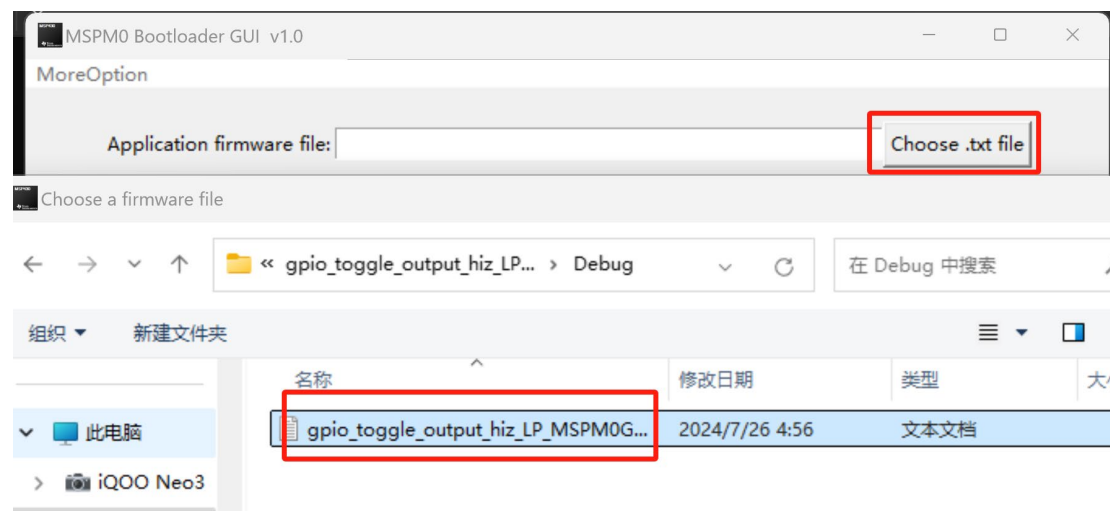
接下来打开该项目的主函数代码文件，该项目中为“gpio_toggle_output_hiz.c”文件，进行修改以匹配 SysConfig 中对功能的设置，然后点击 Project->Build Project，或者直接点击图标即可开始自动从 SysConfig 生成 ti_msp_dl_config 库（该库即为从 SysConfig 配置自动生成的相关功能的初始化配置代码），并编译项目，并生成用于 BSL 烧录的固件文件。如果一切正常，那么将在命令行窗口中看到其在项目文件夹底下的 Debug 文件夹中成功生成命名为 gpio_toggle_output_hiz_LP_MSPM0G3507_nortos_ticlang.txt 固件文件的消息。



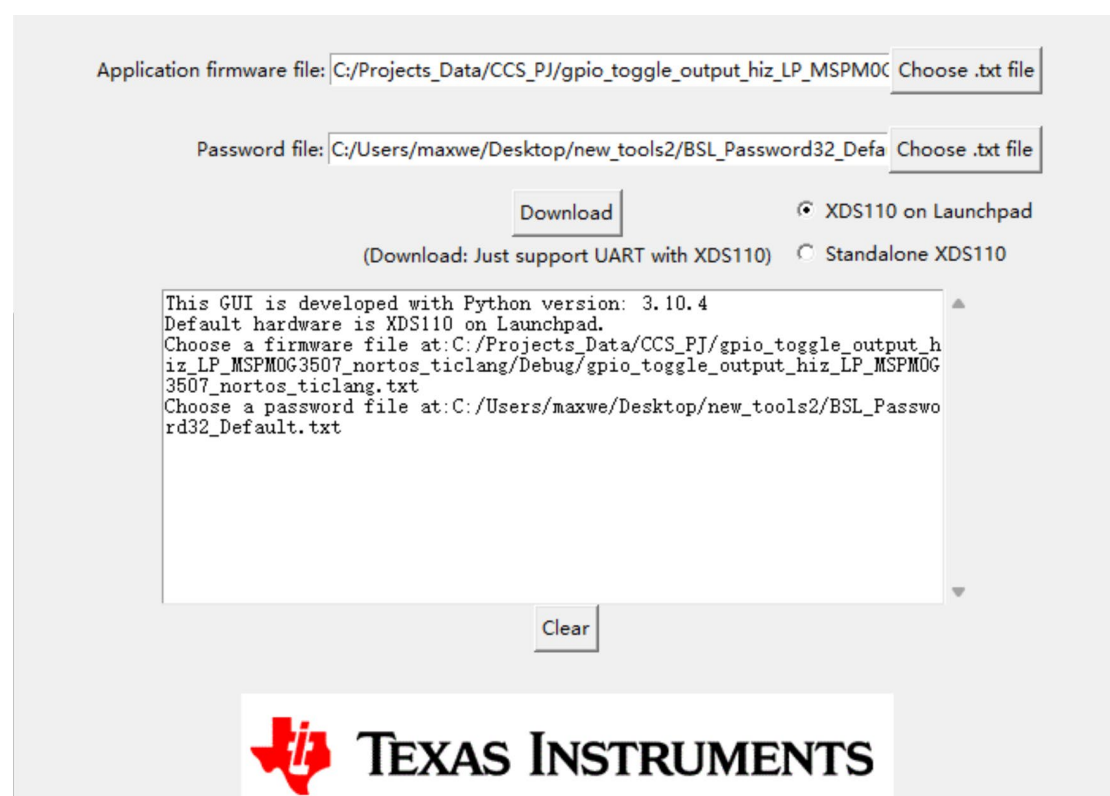
4.2 BSL 下载工具烧录程序

打开下载下来的 BSL 下载工具，在“BSL_download_tool2”中，双击打开 MSPM0_BSL_GUI.exe 程序文件，点击第一个 Choose .txt file 按钮，进入 BSL 固

件文件的选择界面，进入到你自己设置该项目的目录中的 Debug 文件夹，选择刚刚生成的 gpio_toggle_output_hiz_LP_MSPM0G3507_nortos_ticlang.txt 固件文件

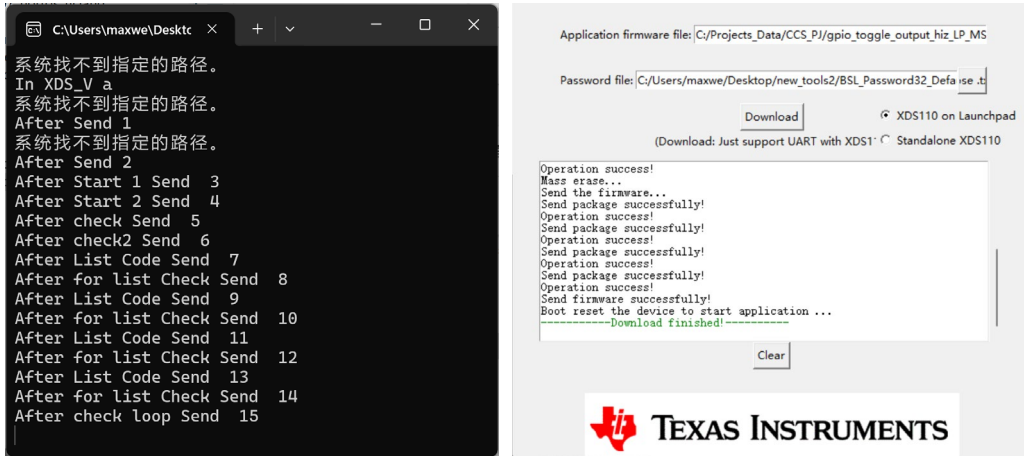


接下来在 Password file 选择框中，同样点击 Choose .txt file 按钮，直接选择弹出的窗口中的 BSL_Password32_Default.txt 文件。完成上述操作后，可以看到工具中显示如下



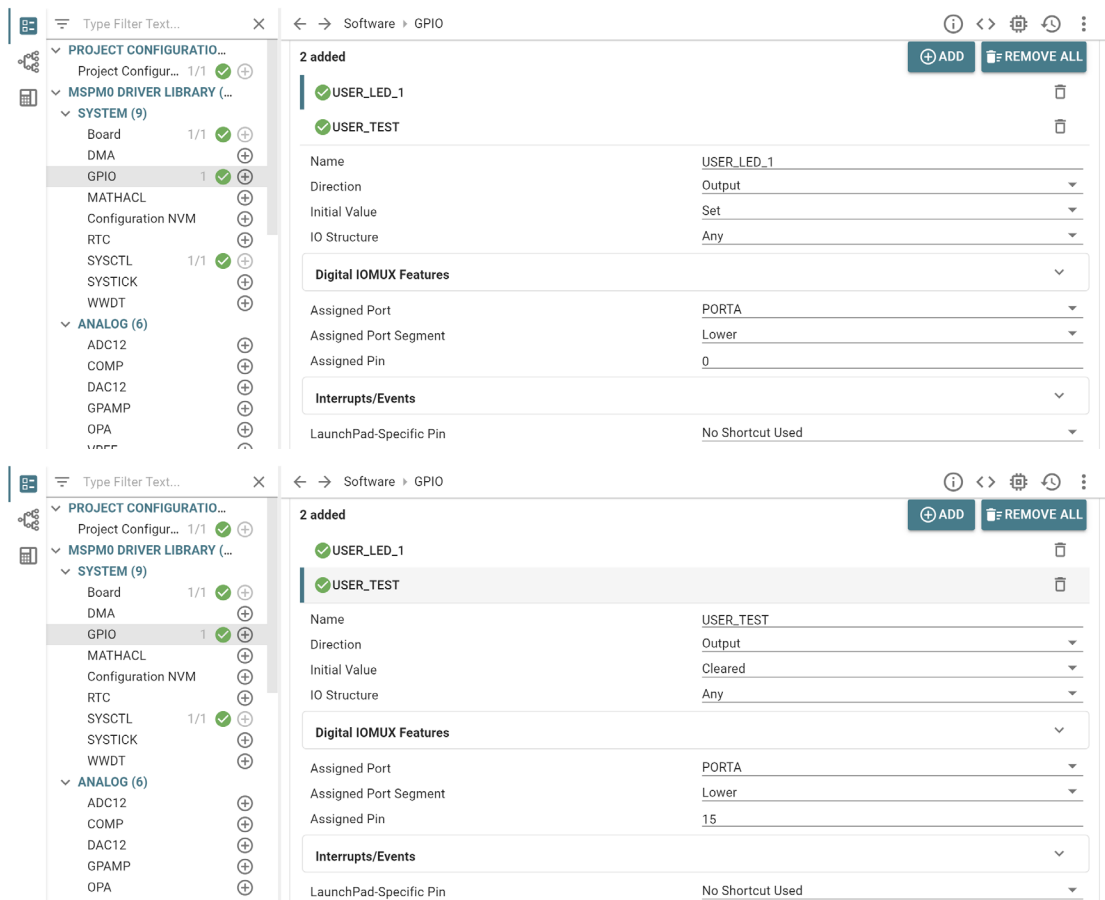
此时将我们的 MSPM0G3507 Eval-Board 通过 USB Type-C 数据线连接电脑，然后按住 RST 按键的情况下按下 BSL 按键，然后先松开 RST 按键，再松开 BSL

按键,此时板子进入了 BSL 固件下载状态,再点击 BSL 下载程序的“Download”按键,正常情况下,命令行界面将开始滚动固件下载进度信息,并经过一段时间后下载完成,下载程序状态输出窗口会显示“Download finished!”这意味着程序成功下载到了我们的核心板上。



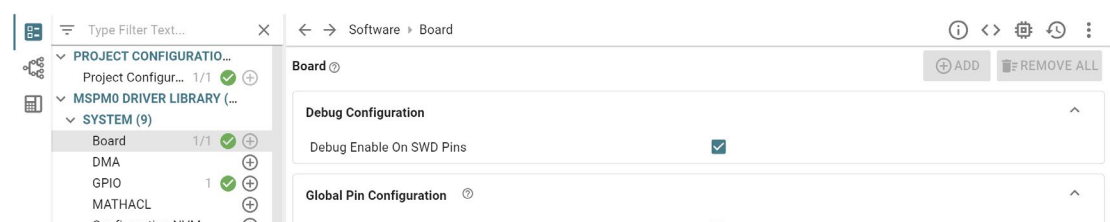
4.3 功能验证

根据 SysConfig 可以看到,该程序设置了两个 GPIO 的功能

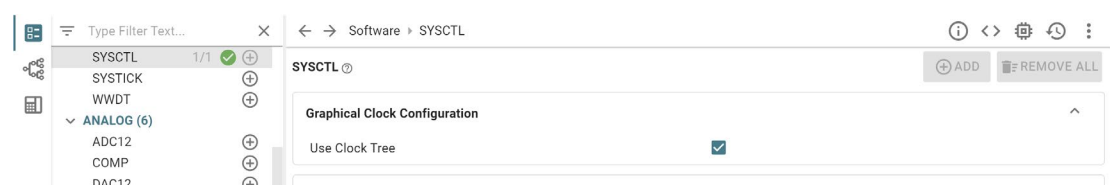


将 GPIO 的 PA0 设置成了输出,并且预设管脚电平为高电平,且在程序中将

该管脚命名为 USER_LED_1；同理将 PA15 设置成了输出，并且预设管脚电平为低电平，且在程序中将该管脚命名为 USER_TEST，



Board 这边勾选了 Debug Enable on SWD Pins，意味着 SWD 管脚功能启用，但是我们这里并没有用到。



SYSCTL 系统控制页中勾选了使用时钟树



时钟树页面中可以看到该项目没有使用的是内部的 RTC 晶振以及内部的系统晶振，且系统晶振工作在 32MHz，最后的 CPU 频率是 32MHz。

```
int main(void)
{
    /* Power on GPIO, initialize pins as digital outputs */
    SYSCFG_DL_init();

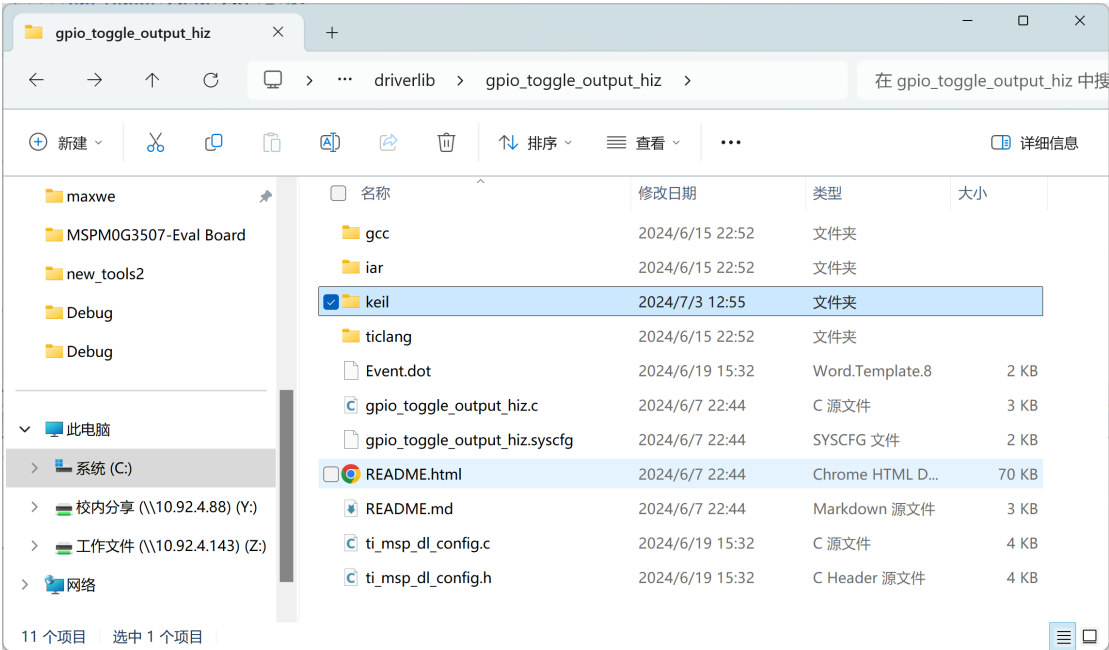
    while (1) {
        /*
         * Toggle the Hi-Z pin and on board LED
         * The pin toggles if an external pull-up is connected, otherwise
         * it will be Hi-Z when high.
         */
        delay_cycles(10000000);
        DL_GPIO_togglePins(GPIO_LEDS_PORT,
                           GPIO_LEDS_USER_LED_1_PIN | GPIO_LEDS_USER_TEST_PIN);
    }
}
```

最后看代码可以了解到该代码是以 10M 个周期进行延时，然后执行 PA0 和 PA15 的管脚电平翻转。由于我们的开发板上没有与这两个关键相连的 LED 灯，因此可以通过万用表或者示波器观测其是否按照预想的进行电平周期翻转。

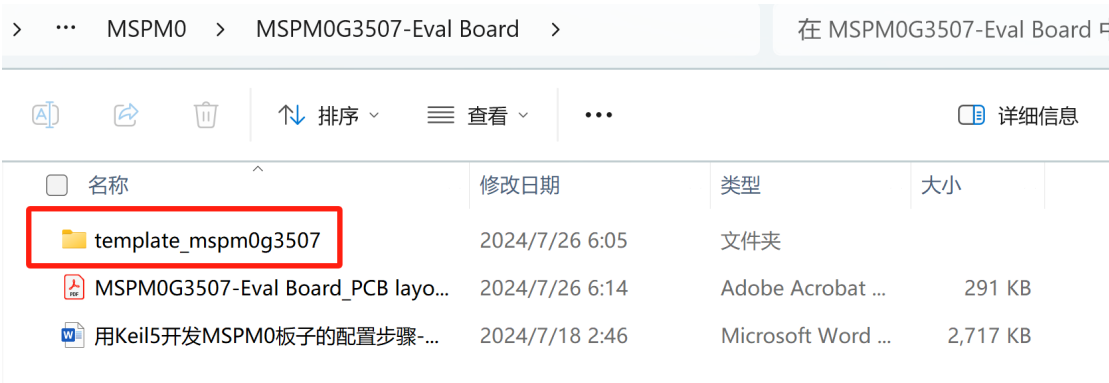
五、基于 Keil 项目模板的构建及配置

正如上一章了解到的，M0SDK 中有很多例程，并且注意到每个例程都会有 4 种工程项目，其中就有 Keil 的项目。但是需要注意的是，官方例程的 Keil，是

假定你直接在 Examples 文件夹中打开 Keil 项目工程并直接进行工程的使用的。因此如果你将项目复制粘贴到别处，非常大概率是无法正常编译使用的！



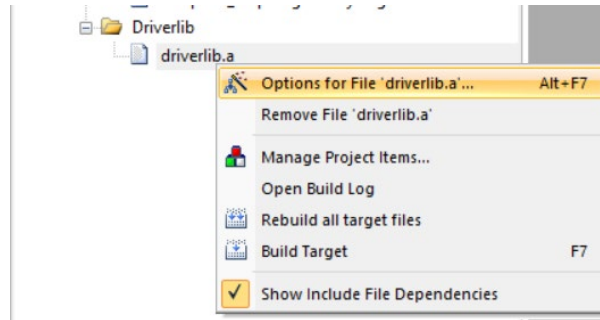
另外官方其实也提供完全空白的 Keil 工程模板，在“M0SDK 安装目录”\examples\nortos\CUSTOM_BOARD\driverlib 中具有所有的 M0 芯片的工程模板。在此需要强调，正如利用 CubeMX 自动生成 Keil 项目一样，利用 Keil 开发 TI 的 MSPM0 芯片，**同样不建议直接从零开始新建工程**，最好是基于一个配置好的模板，生成副本并修改。



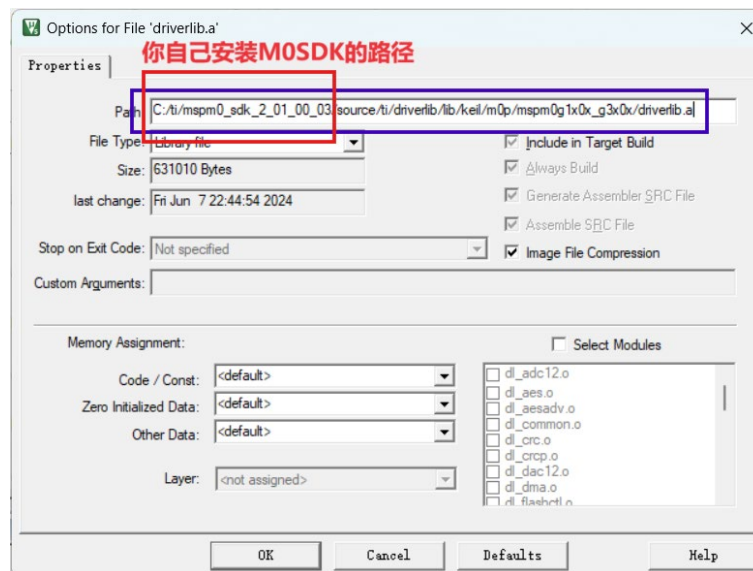
我在 ftp 上提供给大家的面向 MSPM0G3507 的 Keil 工程模板，地址为 ftp:\\10.92.4.88\\电赛培训\\嵌入式开发板+模块资料\\MSPM0\\MSPM0G3507-Eval Board，以此模板为例子，向大家介绍如何在该模板的基础上正确修改，使得模板适配自己的开发环境，拥有一个完美适配自己开发环境的面向 G3507 的 Keil 工程模板。

5.1 修改 M0 的驱动库文件的路径

将项目复制 xia 后，直接打开 ./keil/ template_mspm0g3507.uvprojx 的 Keil 工程项目文件。左侧项目目录界面中的 driverlib.a 所对应的路径大概率与你自己电脑中的路径不一致，需要查看确认修改。首先右键打开该文件的 Option 选项。

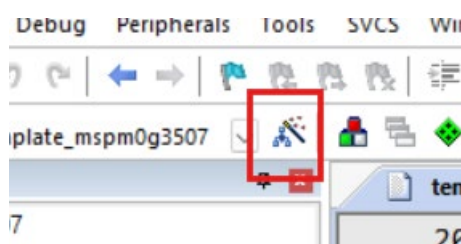


在路径栏替换上你自己的 M0SDK 安装的路径下对应的 driverlib.a 的路径，正常来讲只需要把红框部分的路径修改成你的 M0SDK 文件夹路径即可，后边 /source.....的路径结构一致，无需修改。当然为了保险起见也可以自己按照路径进入到相应的文件夹底下确认是否有 driverlib.a 这个库文件。针对不同的板子的 SDK 有所不同，如果是其他芯片例如 MSPM0L1306，则倒数第二项就不是图示的 mspm0g1x0x_g3x0x 了。不过我们下发的板子都是 MSPM0G3507，所以这里跟着说明图的指引只修改红框的即可。

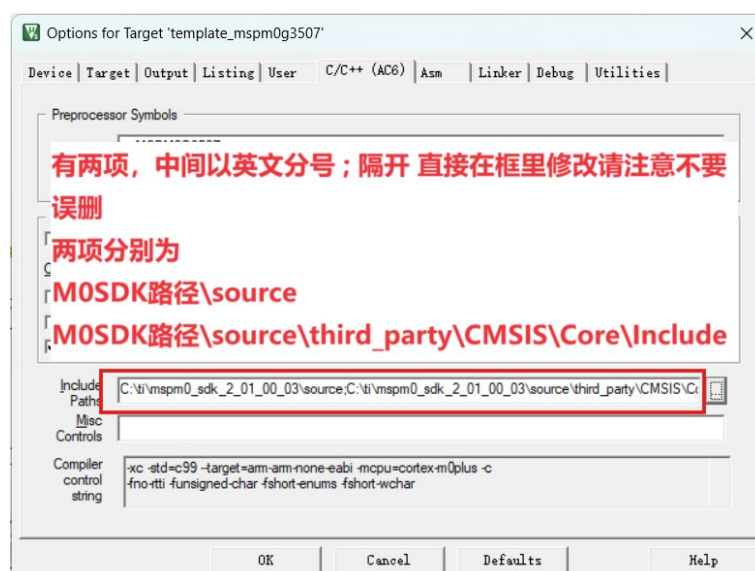


如果没有正确配置好，相关的文件前的图标上会有黄色感叹号的标识🚩，一旦路径修改正确能被正确引用时，黄色感叹号消失，可利用这一点验证是否正确修改。

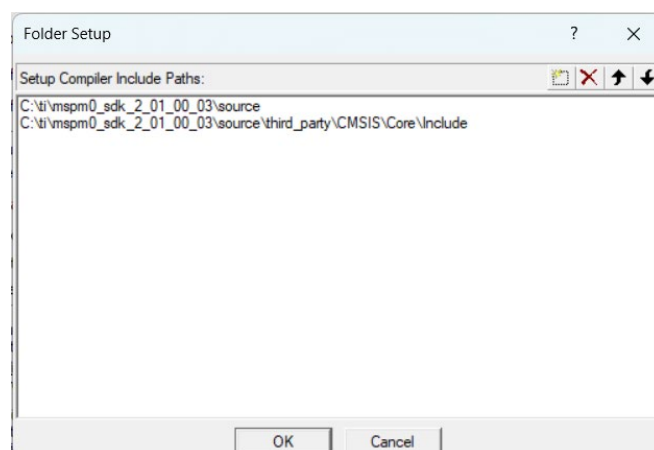
5.2 修改项目中的包含路径 Include Paths



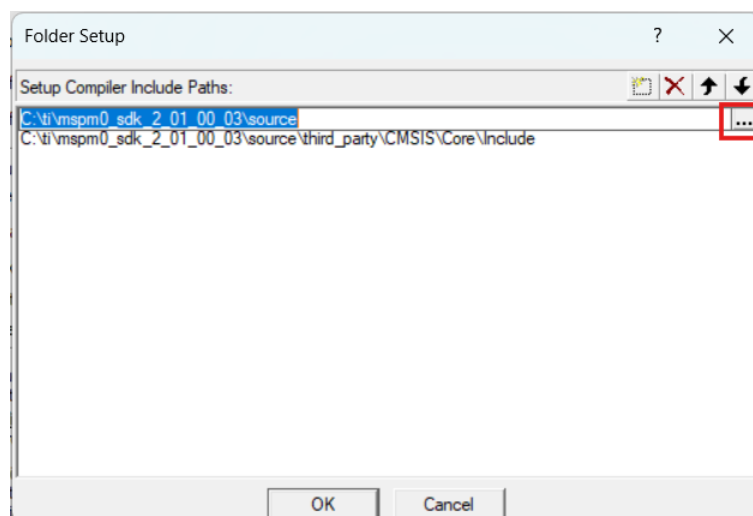
点击“Options for Target”工具，可以弹出工程项目的配置页面，再点击“C/C++ (AC6)”分标签页，可以进入 C/C++ 编译选项页面，修改确认其中的“Include Paths”。该选项能够设置代码中那些缺省地、未按照系统中完整的路径书写那些被 include 的库的头文件所在全部文件夹目录，此处我们只有两项，同样需要根据你自己 MSPM0 的 SDK 安装的路径进行修改。



可以直接点击框框里进行修改，也可以点击右侧的  展开以列表的形式看到两个路径。

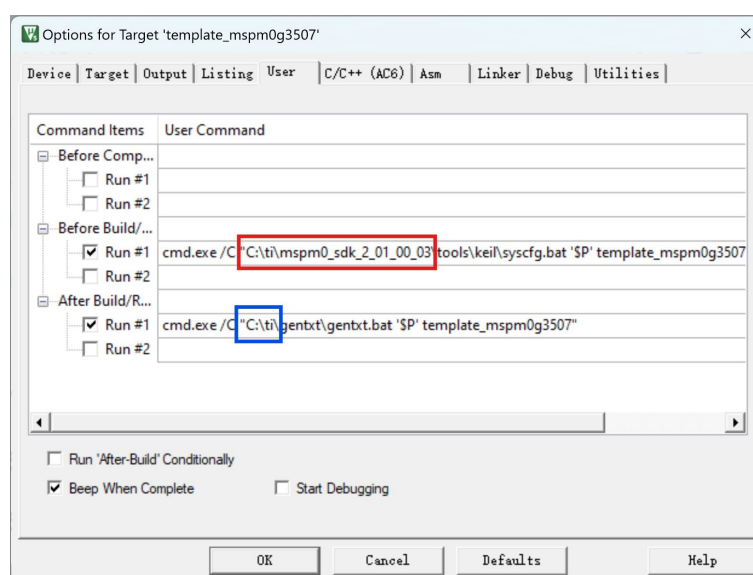


双击某一项同样可以进行修改,此时也可以通过点击双击之后该项最右出现的图标进入图形化选择目录路径的界面进行修改。



5.3 修改项目中的编译前后的执行命令

同样在“Options for target”中,进入“User”标签页,可以看到有 Before Build 和 After Build 两个项目已经填写了相关的命令行指令。Before Build 的命令其实就是在项目构建前,需要调用 SysConfig 的命令行脚本,并且将生成的 ti_msp_dl_config 库自动添加到和核心 C 文件同个路径下,使得核心 C 文件中能够调用该库进行进一步的函数调用;而 After Build 则是在成功将代码生成 axf 文件之后,利用 TI 的 txt 固件文件转换工具,利用命令行调用之前已经下载备好的 gentxt 脚本,实现 txt 固件文件的生成,能够和 CCS 一样,在成功构建之后在 ./keil/Objects 中生成固件 txt 文件,以便于利用 BSL 下载工具进固件下载。

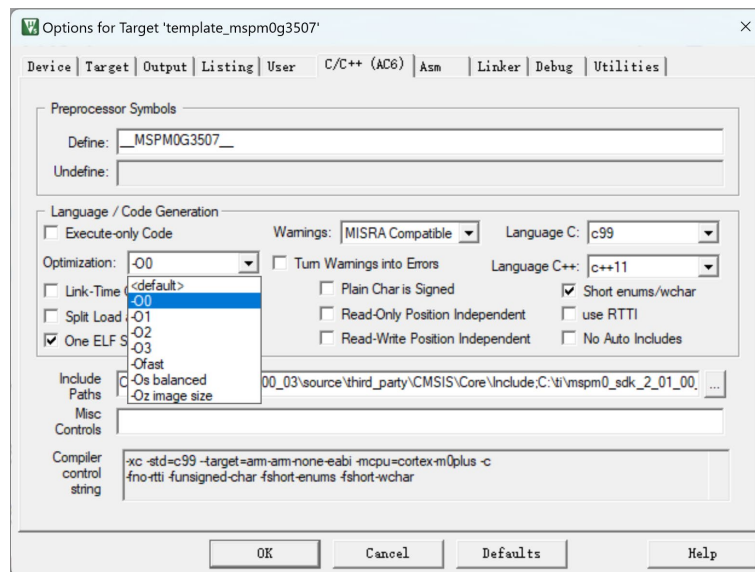


这里需要确认和修改的内容和前文类似，需要将 **Before Build** 中的红框部分路径修改成你自己的开发环境中安装 **M0SDK** 的目录路径地址，例如我的是 `C:\ti\mspm0_sdk_2_01_00_03`；在 **After Build** 中，则需要确认蓝框中的路径地址是你存放从 `ftp` 上下载的 `gentxt` 工具脚本的位置。

5.4 其他注意事项

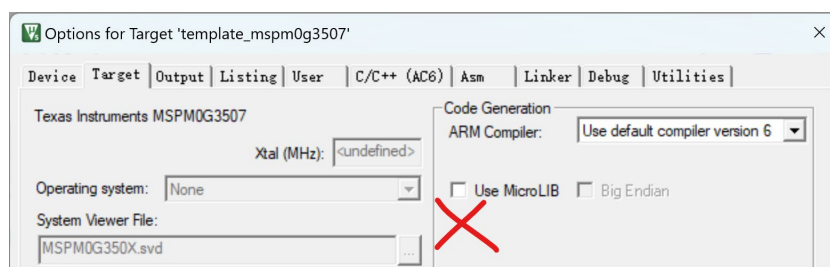
5.4.1 程序优化程度会影响断点调试

如果你需要打断点进行硬件调试，那么请确认同样在“Options for target”中，“C/C++(AC6)”分页中，**Optimization** 一定要选在 **-O0**（24.8.10 更新：根据 24 年电赛时大家的实际测试，有些版本的 Keil 在 **-O0** 的优化选项下，程序无法正常下载运行！因此如果发现例程下载之后不工作，需要将优化选项改成 **-O1**），否则无论是库的代码或者是你写的代码，都可能会被优化算法优化掉，进而导致无法在你打的语句断点实现暂停调试！



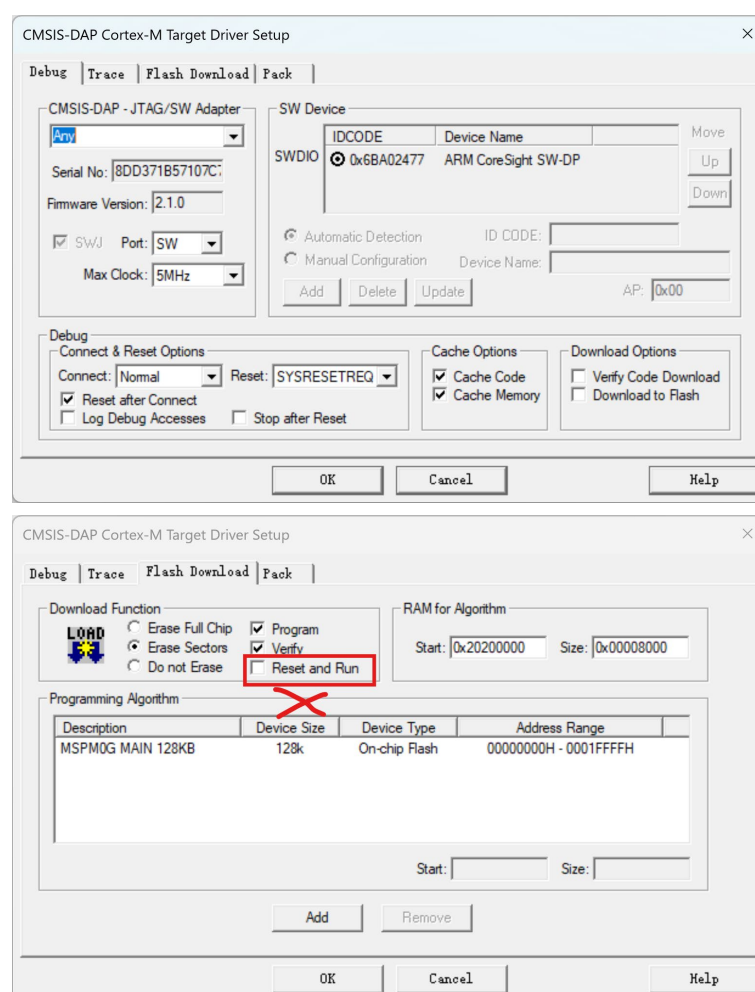
5.4.2 MSPM0 无法适配 MicroLIB

在“Options for target”中，“Target”分页中，右侧的 **Code Generation** 底下有个 **Use MicroLIB**，千万不可勾选上，否则会导致程序无法通过 Keil 烧录到 MSPM0 芯片上！



5.4.3 调试器不可设置下载后自动 Reset and Run

在“Options for target”中，“Debug”分页中，请确认右侧的调试器选择下拉框中，选择的是 CMSIS-DAP Debugger，从而与我们使用的 PWLink2 Lite 相匹配。进一步的，在插入 PWLink2 Lite 之后，点击右边的“Setting”，正常情况下能够查看到调试器的信息，在该页面下选择“Flash and Download”分页，可以对该项目下点击“Load”按钮下载程序时具体的操作进行定义修改，此处请注意千万不要勾选“Reset and Run”！

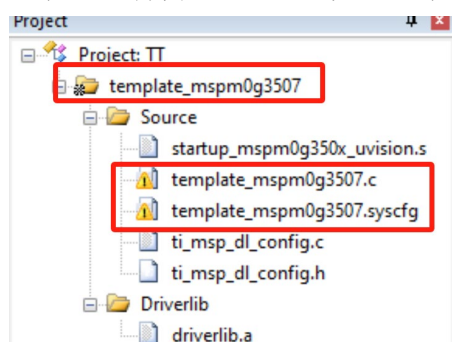


在大部分的开发中，该选项在烧录完成后后，自动重启 MCU 并开始跑新的程序。但是该选项在 TIMSPM0 芯片下，经实验发现，该操作后的第一次重新运行，系统运行速度似乎降低了一半；断电重新上电或者硬件重置等下一次以及未来每一次运行，运行速度又恢复正常了。该现象发现自 delay_cycles()函数的延时效果的翻倍，具体内部机制几何尚不明确，但是请大家规避该操作，不要勾选该选项，每次程序烧写完手动按下 RST 按键给予硬件复位从而使其运行新的程序。

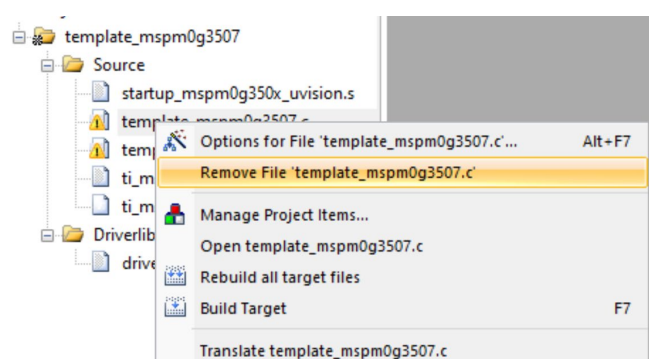
六、基于 Keil 的开发流程

6.1 模板工程信息修改

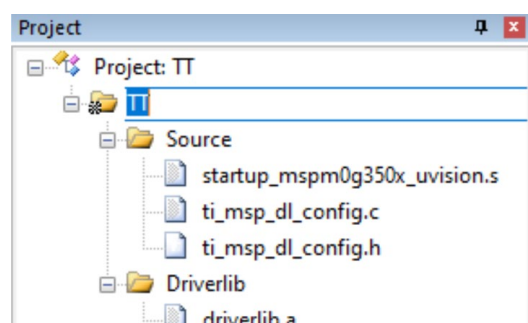
首先，复制一份 MSPM0G3507 的 Keil 模板工程，修改相关文件的文件名，如“template_mspm0g3507.c”、“template_mspm0g3507.syscfg”、“template_mspm0g3507.uvprojx”，“template_mspm0g3507.uvoptx”到新的工程名字，此处以 TT 为例做新的工程名字，那么上述 4 个核心文件直接手动重命名好，然后打开重命名后的新工程“TT.uvprojx”。可以看到项目工程文件并不会自动改名字，我们需要删除这些已经更改了名字的文件并重新引入改了名字的文件。



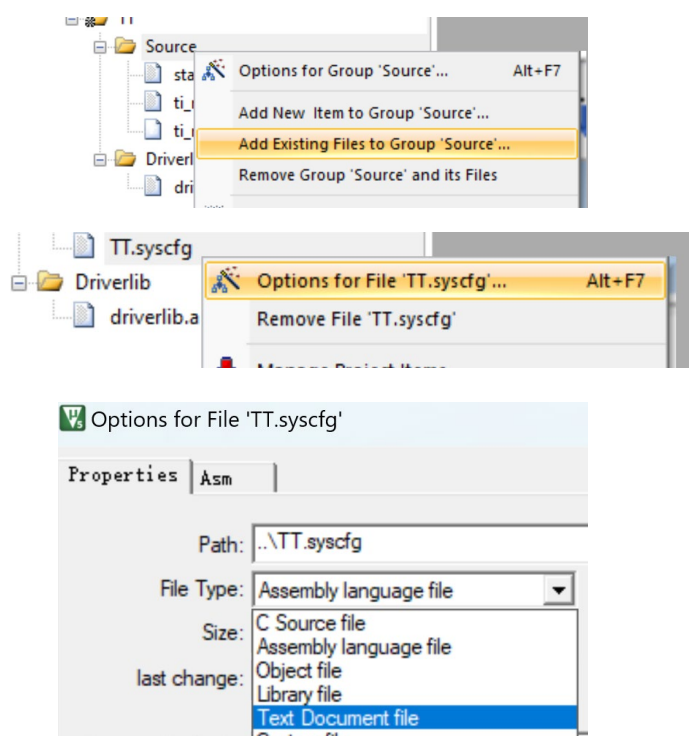
将这些黄色感叹号的文件右键 Remove 删除掉



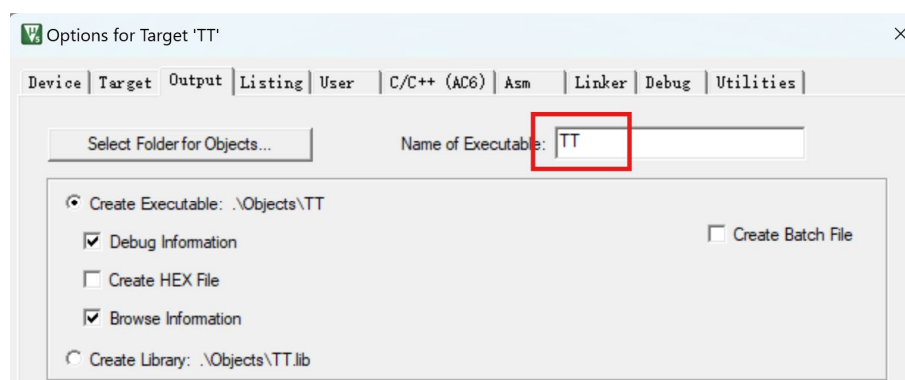
点击项目文件夹名称，按下键盘“F2”同样将其重命名为“TT”



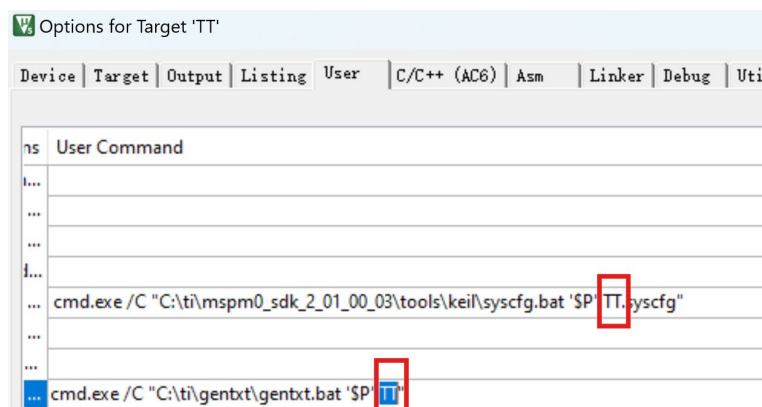
重新添加“TT.c”“TT.syscfg”到项目的 Source 分项下，并将 **TT.syscfg 文件的属性修改为 Text Document file**，这一步非常非常重要，如果没有修改，工程是无法正常构建生成的！



“Options for Target”里“Output”分页的Name of Executable 修改为 TT



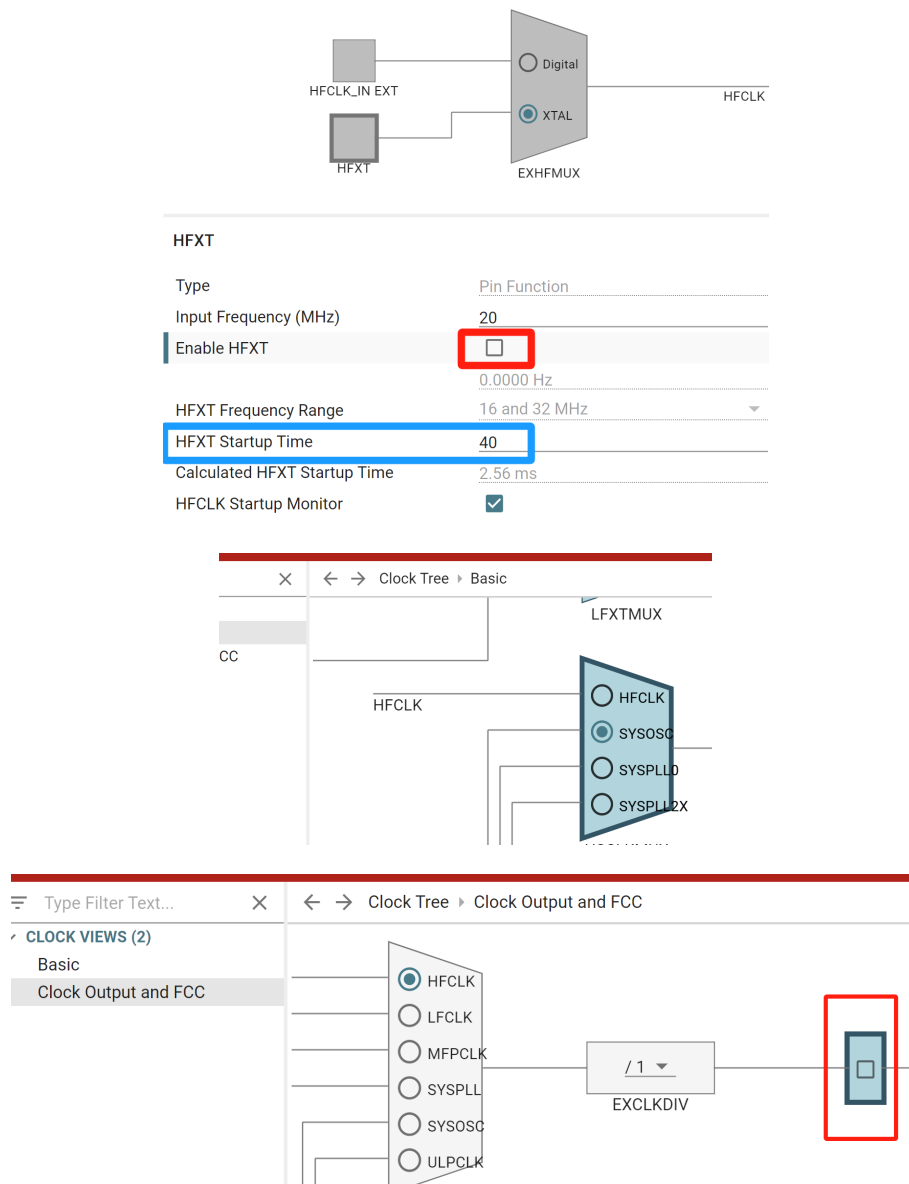
在“User”分页中的命令行，同样需要将原来的“template_mspm0g3507”字样修改成“TT”



至此完成从模板工程更改成新的名字的工程的操作

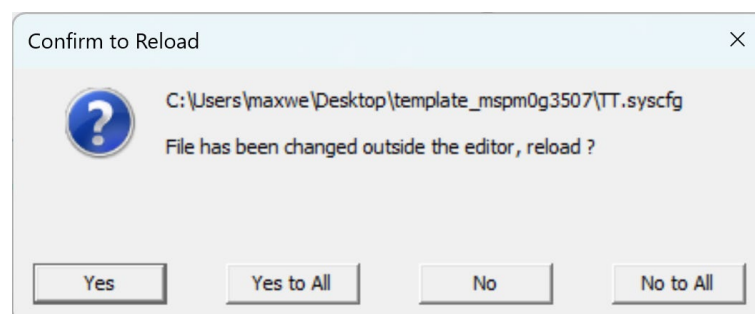
6.2 Keil 中的 SysConfig 管脚功能配置

打开现在的新的叫做 TT 的 Keil 项目，从左侧的项目资源窗口中打开“TT.syscfg”文件，确保右边的编辑界面停留在该文件的前提下，点击 Tools->Sysconfig v1.20.0 - MSPM0 SDK v2_01_00_03，调用 3.4.2 我们为 Keil 安装的 SysConfig 插件，从而调用 SysConfig 软件以打开 TT.syscfg 文件，从而能够进行新的工程的 G3507 芯片的功能配置。此处我们禁用外部晶振，将系统时钟切换到芯片内部晶振，并且关闭时钟树中的 CLK_OUT 时钟信号输出功能。



注意到原本使用外部晶振时，除了设置晶振频率以及使能，蓝框里还设置了启动时间，该数值非常重要，不可以太小，否则会导致外部晶振不工作而导致没有系统时钟进而死机，因此此处最好一直保留着原本的 40 不要改动！

同理可以继续修该其他功能配置，修改好了 SysConfig 文件，点击 File->Save 或者 Ctrl+S 保存后再关闭 SysConfig 软件，此时 Keil 会弹窗通知刚刚打开的 TT.syscfg 文件在别处被修改了，询问你是否重新加载，点击 Yes 即可。



6.3 项目构建生成

管脚功能配置完成之后，可以先进行一次“Build”  将刚刚通过 SysConfig 配置好的芯片的功能生成为 ti_msp_dl_config 库文件，感兴趣的话可以打开该库文件，大致了解 TI 的 MCU 的 HAL 开发下初始化的方法。

接下来就可以对 TT.c 进行代码修改，如果需要添加其他的库文件，同前边添加 TT.c 到项目里一样的操作即可。最后完成了代码之后，可以点击“Build”  直接生成程序文件后，在接入 PwLink2 Lite 调试器的前提下直接烧录，也可以点击  进入硬件调试进行代码调试。

6.4 BSL 镜像文件烧录

每次 Build 完之后，由于我们项目里已经配置好了后处理的 txt 固件生成脚本，因此可以在项目文件夹底下的 ./keil/Debug 中找到最后一次成功 Build 的代码所对应的程序镜像文件，进而借助 BSL_Download_tool2 进行烧录。